

Nucleus Decomposition Revisited: Counting-Based Indexing and Differential Path Maintenance

WENQIAN ZHANG, The University of New South Wales, Australia

ZHENGYI YANG*, The University of New South Wales, Australia

DONG WEN, The University of New South Wales, Australia

YI DING, The University of New South Wales, Australia

WENJIE ZHANG, The University of New South Wales, Australia

XUEMIN LIN, Shanghai Jiao Tong University, China

Nucleus decomposition is a general model for finding cohesive substructures in graphs. It extends k -core and k -truss to higher-order (r, s) -nucleus. Existing algorithms have a clear bottleneck. They explicitly enumerate all s -cliques. The number of s -cliques grows combinatorially with s . In this paper, we revisit nucleus decomposition through two ideas. The first is a counting-based *Clique Path Index* (CPI). It avoids explicit s -clique enumeration. The second is a new *differential path contribution maintenance* view. It explains how to update exact supports during peeling. The CPI encodes the s -clique search space as compact paths. Under differential path maintenance, the support of each active r -clique is the sum of path-local contributions. After each peeling round, we only update paths incident to removed cliques. This view gives exact update rules for three regimes. For $r=1$, we use counter-only updates. For $r=2$, we use closed-form rules and analytical splits. For $r \geq 3$, we use dead-path-aware lazy maintenance. Together, these results support exact nucleus decomposition and hierarchy construction without explicit s -clique materialization. Experiments on real-world datasets show clear gains over prior methods. On comparable settings, our method is about $4\times$ faster than ARB and about $13\times$ to $27\times$ faster than the ND baselines. On some instances, the speedup exceeds two orders of magnitude. Many baseline runs also time out as s grows. Our method still scales to larger s and denser graphs.

CCS Concepts: • **Theory of computation** → **Graph algorithms analysis**; • **Information systems** → **Graph-based database models**.

Additional Key Words and Phrases: nucleus decomposition, clique counting, differential maintenance, cohesive subgraphs, hierarchy

ACM Reference Format:

Wenqian Zhang, Zhengyi Yang, Dong Wen, Yi Ding, Wenjie Zhang, and Xuemin Lin. 2026. Nucleus Decomposition Revisited: Counting-Based Indexing and Differential Path Maintenance. *ACM Trans. Datab. Syst.* 1, 1 (April 2026), 32 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Graphs are fundamental data structures consisting of vertices and edges that model relational information. They are widely used across diverse domains, including social network analysis, bioinformatics, and knowledge graph modeling. Beyond pairwise relations, higher-order structures such as cliques and nuclei often provide deeper insights into the underlying organization of networks, revealing cohesive communities and latent functional modules [5, 35]. In social networks, cliques capture tightly connected friendship groups [26, 53]; in biological systems, dense patterns correspond to protein-protein interactions and functional complexes [5]; in collaboration networks,

*Zhengyi Yang is the corresponding author.

Authors' Contact Information: Wenqian Zhang, The University of New South Wales, Sydney, Australia, wenqian.zhang@unsw.edu.au; Zhengyi Yang, The University of New South Wales, Sydney, Australia, zhengyi.yang@unsw.edu.au; Dong Wen, The University of New South Wales, Sydney, Australia, dong.wen@unsw.edu.au; Yi Ding, The University of New South Wales, Sydney, Australia, yi.k.ding@unsw.edu.au; Wenjie Zhang, The University of New South Wales, Sydney, Australia, wenjie.zhang@unsw.edu.au; Xuemin Lin, Shanghai Jiao Tong University, Shanghai, China, xuemin.lin@sjtu.edu.cn.

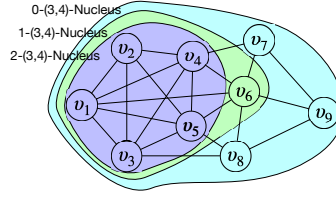


Fig. 1. An example of a graph G .

they represent research groups formed through co-authorship [16, 47]; and on the World Wide Web, hyperlinks induce dense subgraphs corresponding to related content or topical communities [14, 31].

Decomposing graphs into hierarchically cohesive substructures is a fundamental task in graph mining to understand complex network, enabling diverse applications such as community detection [15], influence propagation [20], anomaly detection [2], graph visualization [37], and protein modeling [34]. Common graph decomposition models include the k -core [4, 30] and k -truss [12, 67], which recursively peel away low-degree vertices or edges to expose progressively denser subgraphs. To better capture higher-order dense structures, *nucleus decomposition* was introduced [41], providing finer granularity that uncovers progressively denser and more cohesive subgraphs in real-world networks. It generalizes classical models such as k -core and k -truss by defining cohesiveness in terms of r -cliques and their participation in larger s -cliques, and can be naturally extended to other higher-order structures.

Specifically, given two integers $0 < r < s$, an (r, s) -nucleus is defined as a maximal subgraph in which every r -clique participates in at least k distinct s -cliques (and all such r -cliques are s -connected). The nucleus core number $\kappa_s(R)$ of an r -clique R is defined as the maximum k such that R belongs to a k -(r, s)-nucleus. This generalization subsumes classic notions: the $(1, 2)$ -nucleus corresponds to the k -core [43], the $(2, 3)$ -nucleus corresponds to the k -truss [66], while higher-order (r, s) values yield clique-based, richer structural representations of network cohesiveness.

Example 1.1. Figure 1 illustrates a sample graph G . Taking the $(3, 4)$ -nucleus as an example, the subgraph induced by vertices v_1, v_2, v_3, v_4 , and v_5 forms a 2-(3, 4)-nucleus, as each 3-clique is contained in at least two 4-cliques. For instance, the 3-clique (v_1, v_2, v_3) belongs to the 4-cliques (v_1, v_2, v_3, v_4) and (v_1, v_2, v_3, v_5) . Similarly, all other 3-cliques in this subgraph satisfy this condition. The maximum nucleus core number for the $(3, 4)$ -nucleus in this graph is 2.

Applications. Compared with coherent subgraph models such as the k -core and k -truss, the (r, s) -nucleus provides a more general higher-order model, where (r, s) control cohesiveness. In particular, r specifies the base clique unit, while s specifies the cohesiveness of the resulting nuclei and thus how tightly connected they are. A larger s imposes a stricter support requirement: an r -clique must participate in many distinct s -cliques, so nuclei persist only when higher-order overlap is abundant rather than accidental, providing stronger evidence of cohesion [41]. This requirement arises naturally in many domains; we discuss representative examples below:

Coordinated fraud / collusion. From interaction logs, we can build a graph where suspicious accounts tend to act in lockstep on many targets. High- s nuclei help isolate small coordinated rings by filtering out casual triangles and keeping only groups with repeated multi-way coordination [6, 17].

Recommendation / e-commerce. In an item-to-item graph from co-purchase or co-click signals, high- s nuclei highlight compact bundles of items that co-occur again and again across many users, which can be used as stable and interpretable bundles for browsing, diversification, or cold-start organization [27].

Collaboration analysis. In a coauthor graph, high- s nuclei capture stable teams whose collaborations are supported by many different higher-order coauthorship cliques, instead of being connected only through one large paper or a single hub author [33].

Existing Methods and Key Limitation. Despite its effectiveness and broad applicability, computing nucleus decomposition is computationally intensive, motivating extensive research on improving its efficiency [32, 40–42, 44, 45]. Existing studies primarily focus on optimizing the peeling process [44, 45], where the key idea is to iteratively remove low-support r -cliques those that participate in fewer s -cliques to expose progressively denser substructures. Other lines of research explore parallel acceleration, either through local update strategies [40] or multi-threaded peeling algorithms [44, 45].

However, all existing algorithms share a fundamental limitation: they rely on explicit **clique enumeration**, requiring all s -cliques in the graph to be enumerated. Such enumeration significantly limits scalability, as the number of s -cliques increases combinatorially with s , often reaching trillion scale even for moderate values of s . As a result, existing approaches are typically limited to small parameter settings (e.g., (3, 4)-nucleus) or to small and sparse graphs. In theory, the best existing methods achieve asymptotic work complexity comparable to full s -clique enumeration [44, 45], leaving the intrinsic computational barrier of explicit clique listing unresolved, particularly for larger graphs and higher parameter values.

Example 1.2. The number of k -cliques grows explosively with increasing k . We observe that in DBLP, the number of 6-cliques already exceeds 4×10^9 and reaches 1.5×10^{36} at $k=57$; in WEB-STANFORD, it includes 2.2×10^{12} 28-cliques. Because existing frameworks explicitly enumerate s -cliques, the computation becomes quickly infeasible—even enumerating all 6-cliques alone incurs substantial overhead (Figure 12).

Research Question. This leads to the critical research question and the main objective of this paper: **How to compute (r, s) -nucleus decomposition without explicitly enumerating s -cliques?**

Challenges. Avoiding s -clique enumeration in nucleus decomposition is highly challenging, as enumeration is deeply embedded in all existing algorithmic frameworks. All existing algorithms rely on explicitly enumerating all s -cliques for three essential reasons:

Computing the support of each r -clique. The support of an r -clique R is defined as the number of s -cliques containing it. Existing methods compute this by enumerating all s -cliques and recording, for each r -clique, the set of s -cliques that include it.

Updating neighboring r -cliques during peeling. When an r -clique R is removed during the peeling process (along with all s -cliques containing it), every r -clique R' that shares an s -clique with R must have its support decremented by 1. This update requires knowing, for each s -clique, the set of r -cliques it contains; thus, adjacency among r -cliques is derived from enumerated s -cliques.

Defining connectivity for hierarchy construction. Two r -cliques R and R' are s -connected if a sequence of r -cliques exists such that each consecutive pair of cliques shares at least one common s -clique. To identify nucleus components for hierarchy construction, existing algorithms enumerate and traverse all s -cliques to determine these connectivity relationships.

Our Idea. To efficiently support the above core operations of nucleus decomposition while avoiding explicit s -clique enumeration, we design a novel auxiliary data structure, the *Clique Path Index* (CPI). CPI encodes the space of s -cliques into compact paths, enabling counting, updates, and connectivity queries to be executed directly over these paths without enumeration. The proposed index provides the following key features:

Counting without enumeration. CPI enables s -clique counting and the computation of r -clique supports by aggregating path statistics, thereby avoiding the exponential overhead of enumerating all s -cliques as in existing algorithms.

Dynamic and efficient updates. CPI performs efficient incremental updates during peeling by dynamically modifying or splitting affected clique paths upon deletions. As clique information is aggregated in the path representation, removing a single r -clique implicitly accounts for the removal of many s -cliques, thereby significantly reducing redundant computation during peeling.

Built-in connectivity maintenance. CPI maintains s -connectivity naturally among r -cliques during updates. Connectivity relationships are preserved and adjusted within the indexed paths, enabling hierarchy construction to be performed directly without additional traversal or re-enumeration.

From index design to differential maintenance. While CPI resolves the representation problem, the remaining algorithmic challenge is how to maintain exact supports during peeling without reverting to local re-materialization. Our journal-level view is that nucleus peeling on CPI is fundamentally a *differential path contribution maintenance* problem: each path contributes a local amount of support to the r -cliques it covers, and after a peeling round only the contributions on paths incident to removed r -cliques can change. This perspective isolates the true source of efficiency gains beyond the index itself and explains why different regimes of r admit different exact update mechanisms.

Contributions. Building on the proposed *Clique Path Index* (CPI), we present a counting-based nucleus decomposition framework. We also present a higher-level differential maintenance theory for exact peeling. The main contributions of this work are summarized as follows:

Counting-based nucleus decomposition via the Clique Path Index. We propose a compact path-based representation of the s -clique search space that avoids explicit s -clique enumeration while still supporting exact support counting, dynamic updates, and hierarchy construction. This is the representation layer of our method.

A differential path contribution maintenance theory. We formalize nucleus peeling on CPI as maintaining path-local support contributions under deletions. Under this view, the support of each active r -clique is the sum of path contributions. After each peeling round, only paths incident to removed r -cliques need updates.

Exact regime-specific update operators for $r=1$, $r=2$, and $r\geq 3$. Under the differential maintenance view, we derive three exact specialized instantiations: counter-only path maintenance for $r=1$, closed-form deltas and analytical single-edge splits for $r=2$, and dead-path-aware lazy maintenance for $r\geq 3$. These instantiations show how the same framework leads to different update operators in different regimes.

Integrated hierarchy construction and comprehensive evaluation. Our method naturally preserves s -connectivity among r -cliques within CPI, enabling hierarchy construction with minimal extra overhead. Experiments show clear gains over prior methods. On comparable settings, our method is about $4\times$ faster than ARB and about $13\times$ to $27\times$ faster than the ND baselines. On some instances, the speedup exceeds two orders of magnitude. The gains hold for both decomposition and hierarchy construction.

Extension over the conference version. Compared with the conference version of this work, this journal version makes three substantive extensions. First, it introduces the differential path contribution maintenance formalism. This formalism isolates the exact support-update primitive beyond the index itself. Second, it derives regime-specific sufficient states and exact update operators for $r=1$, $r=2$, and $r\geq 3$. These operators follow one common maintenance principle. Third, it strengthens the optimization and complexity analysis. It explains how different peeling regimes reduce update costs without losing exactness.

Organization. Section 2 reviews the problem definition and notation. Section 17 revisits the peeling baseline and motivates avoiding explicit s -clique enumeration. Section 3 introduces the representation layer through CPI. Section 4 then lifts it to a differential path contribution maintenance view. Section 5 then instantiates this view for general $r \geq 3$, for $r=2$, and for $r=1$, followed by dead-path-aware refinements. The remaining sections present hierarchy construction, experiments, case studies, and related work.

2 Preliminaries

Graph Notations. We focus on a *simple, unweighted* graph, defined as a pair $G = (V, E)$ where V is the set of vertices and $E \subseteq V \times V$ is the set of undirected edges. We use $N_G(v) = \{u \in V \mid (u, v) \in E\}$ to denote the set of neighbors of a vertex $v \in V$ in G , and $\deg_G(v) = |N_G(v)|$ to denote its degree. We use $G[V']$ to denote the subgraph *induced* by a vertex subset $V' \subseteq V$. We store a graph G in the classical Compressed Sparse Row (CSR) format, consisting of two integer arrays: offsets of length $|V|+1$ and edges of length $|E|$.

Definition 2.1 (k -Clique). A k -clique is a vertex set $C \subseteq V$ with $|C| = k$ such that $(u, v) \in E$ for every $u, v \in C$.

Definition 2.2 ((r, s) -Clique Degree). Given two integers $0 < r < s$, the (r, s) -clique degree of an r -clique R in G is defined as

$$\deg_{(s)}^G(R) = \left| \{ S \subseteq V \mid R \subseteq S, S \text{ is a } s\text{-clique in } G \} \right|.$$

We also refer to $\deg_{(s)}^G(R)$ as the s -clique support of R in G . When the graph G is clear from the context, we simply write $\deg_{(s)}(R)$.

Definition 2.3 (k -admissible s -clique). Given integers $0 < r < s$ and $k > 0$, an induced s -clique S in G is called k -admissible if every r -clique $R \subseteq S$ satisfies $\deg_{(s)}(R) \geq k$.

Definition 2.4 (k -(r, s)-nucleus). Let $\mathcal{S}_k$ be the set of all k -admissible s -cliques of G . For any subset $\mathcal{S} \subseteq \mathcal{S}_k$, denote $G[\mathcal{S}] = G \left[\bigcup_{S \in \mathcal{S}} S \right]$. A subgraph $G_k = G[\mathcal{S}]$ is a k -(r, s)-nucleus if

- (1) G_k is s -connected: for any two r -cliques $R, R' \subseteq V(G_k)$, there is a sequence $R = R_1, \dots, R_t = R'$ with $R_i \cup R_{i+1} \subseteq S_i$ for some $S_i \in \mathcal{S}$;
- (2) G_k is *maximal*, i.e., no proper superset $\mathcal{S}' \supset \mathcal{S}$ still satisfies condition (1).

A k -(r, s)-nucleus is a maximal subgraph in which every r -clique participates in at least k distinct s -cliques, and all such r -cliques are tightly connected through shared s -cliques.

Definition 2.5 (Core Number). Given two integers $0 < r < s$, the (r, s) -nucleus core number (or *coreness*) of an r -clique R , denoted by $\kappa_s(R)$, is the largest integer k such that there exists a k -(r, s)-nucleus G_k in G containing R . Equivalently, it is the largest integer k for which R belongs to some k -(r, s)-nucleus G_k in G .

Problem statement. Given a graph G and integers $0 < r < s$, the goal is to compute the (r, s) -nucleus core number $\kappa_s(R)$ for every r -clique R in G . (The hierarchy construction of k -(r, s) nuclei will be formally discussed in Section 6.)

Computing these core numbers requires tracking, for each r -clique R , its *active* s -clique support $\deg_{(s)}(R)$ (Definition 2.2) under a sequence of deletions. A widely used approach in prior work is the peeling-based nucleus decomposition [39, 41, 42, 44, 45]: starting from the full graph, it iteratively removes the active r -cliques with the smallest current support and updates the supports of the remaining r -cliques. We summarize this baseline next to fix terminology and to expose the scalability bottleneck that motivates our index-based design.

Algorithm 1: Peeling Framework**Input:** Graph G , clique sizes $0 < r < s$ **Output:** $\kappa_s(R)$ for every r -clique R

```

1 Enumerate all  $r$ -cliques in  $G$ ;
2 Enumerate all  $s$ -cliques in  $G$ ;
3 mark every  $s$ -clique  $S \in G$  as active;
4  $\delta(R) \leftarrow$  the number of  $s$ -cliques that contain  $R$ 
5  $coreness \leftarrow 0$ 
6 while there exists active  $r$ -cliques do
7   find  $c = \min\{\delta(R) \mid R \text{ is active}\}$ 
8    $coreness \leftarrow \max(c, coreness)$ 
9   let  $\mathcal{R} = \{R \mid \delta(R) = c, R \text{ active}\}$ 
10  foreach  $R \in \mathcal{R}$  do
11     $\kappa_s(R) \leftarrow coreness$ 
12    mark  $R$  as inactive;
13    foreach active  $s$ -clique  $S$  with  $R \subseteq S$  do
14      mark  $S$  as inactive;
15      foreach active  $r$ -clique  $R' \subseteq S$  do
16         $\delta(R') \leftarrow \delta(R') - 1$ ;
17 return  $\kappa_s(R)$  for every  $r$ -clique  $R \in G$ 

```

Baseline: Peeling-Based Nucleus Decomposition. Algorithm 1 illustrates the peeling paradigm. In the *initial counting* phase, all r -cliques and s -cliques are enumerated and marked active, and each active r -clique R is assigned its initial support $\deg_{(s)}(R)$. In the peeling loop, the algorithm repeatedly removes the active r -cliques with the smallest support; then every active s -clique that contains a removed r -clique is marked inactive, which decreases the support of the remaining r -cliques that it contains. The coreness $\kappa_s(R)$ of each r -clique is the peeling level at which it is removed.

This framework runs in $O(|K_r| + |K_s|)$ time [44], where $|K_r|$ and $|K_s|$ denote the numbers of r -cliques and s -cliques, respectively. However, both time and space critically depend on *explicitly enumerating* s -cliques (during initial counting and during support updates). Since $|K_s|$ can grow combinatorially with s , this dependence quickly becomes prohibitive as s increases, which explains why prior methods typically target only small configurations (e.g., (4, 5)-nucleus) on small or sparse graphs. Local update-based variants have been proposed to improve parallelism [40], but they still rely on s -clique enumeration and thus inherit the same bottleneck. These limitations motivate algorithmic designs that avoid explicit s -clique enumeration.

Baseline Primitive: Eager Support Maintenance. The reference methods in our experiments differ in implementation details. However, they share the same exact peeling skeleton. They repeatedly remove minimum-support active r -cliques. They then maintain exact supports for the surviving r -cliques. The key difference lies in the *support-maintenance primitive*. In prior methods, support maintenance is eager. When a peeled unit invalidates incident s -cliques, the algorithm either enumerates the affected s -cliques or reconstructs the affected local structure. It then recomputes the supports. This primitive is *reconstructive*. It materializes or revisits local clique structure after each update.

From Reconstructive to Differential Maintenance. Our journal-level goal is not to replace the exact peeling skeleton. Our goal is to replace the reconstructive maintenance primitive. The CPI gives a compact counting representation. DPCM shows how to update exact supports by *differential path contribution updates*. That is, we update only the local path states incident to removed units. We do not re-materialize the affected s -clique space. This point is important. Our novelty is not a new decomposition paradigm. Our novelty is a new exact support-maintenance theory under the same decomposition paradigm.

Algorithm 2: (r, s) -CND

Input: Graph $G = (V, E)$, clique sizes $r < s$
Output: Core value $\kappa_s(R)$ for every r -clique R

```

1  $\mathcal{B} \leftarrow \text{BUILDINDEX}(G);$ 
  // initialize  $s$ -support for all  $r$ -cliques
2  $\text{support} \leftarrow s\text{-CLIQUECOUNT}(\mathcal{B}, r, s)$ 
3 build a min-heap  $H_{\min}$  using  $\text{support}$ ;
4  $c_{\min} \leftarrow 0;$ 
5 while  $H_{\min}$  is not empty do
6    $R_{\min} \leftarrow \text{ExtractMin}(H_{\min});$ 
7    $c_{\min} \leftarrow \max(c_{\min}, \text{support}(R_{\min}));$ 
8    $\kappa_s(R_{\min}) \leftarrow c_{\min};$ 
9    $\text{UPDATEINDEX}(R_{\min}, \mathcal{B});$                                 // remove  $R_{\min}$  from  $\mathcal{B}$ 
  // update support of affected  $r$ -cliques
10   $\text{UPDATESUPPORT}(R_{\min}, \mathcal{B}, \text{support}, H_{\min})$ 
11 return  $\kappa_s(R)$  for all  $r$ -cliques  $R$  in  $\mathcal{B}$ ;
```

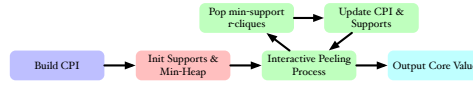


Fig. 2. Overview of our approach.

3 The Representation Layer: Clique Path Index

In this section, we present the representation layer of our method for nucleus decomposition, which addresses the limitations of existing methods by eliminating their reliance on explicit s -clique enumeration. The key idea of our algorithm is to leverage the proposed *Clique Path Index* (CPI) to directly count s -cliques while maintaining and updating the index dynamically during the peeling process. In the following, we first provide an overview of this counting/indexing layer, then review the background on clique counting, and finally present the definition and design of CPI.

3.1 Overview

We observe that a key step in nucleus core decomposition is support computation. For each r -clique, this step only needs the number of s -cliques that contain it. This suggests that explicit enumeration of s -cliques is not necessary. It is both feasible and desirable to avoid it. Based on this observation, we propose a minimal counting-and-indexing layer, as shown in Algorithm 2.

Key Idea. The core idea is simple. We avoid explicit enumeration of s -cliques by using an auxiliary index. Our method still follows the peeling paradigm. The index supports four key operations: (i) **BUILDINDEX**, which initializes the auxiliary index structure (Line 1); (ii) **s -CLIQUECOUNT**, which computes the initial s -support for each r -clique (Line 2); (iii) **UPDATEINDEX**, which removes peeled r -cliques from the index (Line 9); and (iv) **UPDATESUPPORT**, which identifies and updates the supports of affected r -cliques after each removal (Line 10). In the conference version, these four operations were mainly presented from a counting/indexing viewpoint. In this journal version, operations (iii) and (iv) are lifted to the *differential path contribution maintenance* view. **UPDATEINDEX** only changes the local state of affected paths. **UPDATESUPPORT** uses exact differential updates of path contributions. It does not use reconstructive support recomputation. Figure 2 illustrates the overall pipeline of our approach.

Index Support. If an index supports these four operations efficiently, then nucleus decomposition can be performed without enumerating any s -cliques. This removes the main bottleneck. The key journal-level

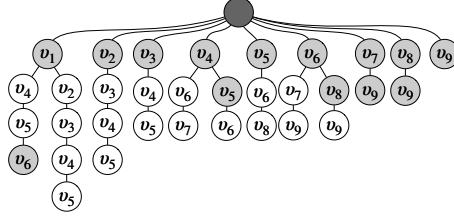


Fig. 3. An example of a Succinct Clique Tree (SCT). White nodes denote pivots; gray nodes denote holds.

refinement is also simple. The index is not only a compact representation of clique space. It also carries the differential maintenance state used for exact support updates.

3.2 Succinct Clique Tree

We first review the state-of-the-art clique counting method, the *Succinct Clique Tree* (SCT) [19]. The SCT compacts the Bron–Kerbosch pivoting search space for exact clique counting. Instead of enumerating all cliques, SCT constructs a hierarchical search tree in which each node corresponds to a state in the Bron–Kerbosch maximal clique search (the pivot-based version) [7, 52], thereby implicitly representing all subcliques. Vertices are processed in *degeneracy order*, and edges are oriented to define $N^+(\cdot)$. For each root vertex v in SCT, every state maintains two sets: V_h (hold, already fixed) and V_p (pivot, eligible extensions). Pivoting on $u \in V_p$ branches only on $V_p \setminus N(u)$ while restricting candidates to $V_p \cap N(\cdot)$. Thus, each *clique path* succinctly represents a family of cliques without enumerating them explicitly. This structure supports aggregated counting of all subcliques (of sizes r , s , etc.), enabling exact clique counting without enumeration.

Figure 3 illustrates the SCT constructed for the example graph shown in Figure 1. We first define a *clique path* as follows:

Definition 3.1 (Clique Path). In a Succinct Clique Tree, a *clique path* $\mathcal{P}$ is any root-to-leaf path. All clique paths are enumerated in left-to-right depth-first order, with the i -th path denoted by $\mathcal{P}_i$. Formally, $\mathcal{P}_i = \langle v_{a_1}, v_{a_2}, \dots, v_{a_\ell} \rangle$, where v_{a_1} is the root, v_{a_ℓ} is the leaf, and ℓ is the path length.

Example 3.1. Figure 3 shows an example of a SCT. $\mathcal{P}_1 = \langle v_1, v_4, v_5, v_6 \rangle$, the second path is $\mathcal{P}_2 = \langle v_1, v_2, v_3, v_4, v_5 \rangle$. There are eleven clique paths in total, and the last path is $\mathcal{P}_{11} = \langle v_8, v_9 \rangle$.

Lemma 3.1 (Unique Path Encoding [19]). *For any k -clique C_k in the graph, there exists only one clique path $\mathcal{P} = V_h(\mathcal{P}) \cup V_p(\mathcal{P})$ such that $V_h(\mathcal{P}) \subseteq C_k \subseteq (V_h(\mathcal{P}) \cup V_p(\mathcal{P}))$. Here, $V_h(\mathcal{P})$ and $V_p(\mathcal{P})$ denote the sets of hold vertices and pivot vertices on the path $\mathcal{P}$, respectively.*

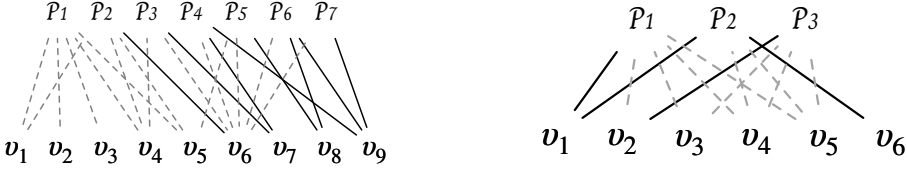
Intuitively, any k -clique represented by a path $\mathcal{P}$ must include all hold vertices of that path, while the pivot vertices are optional. Formally, a k -clique C_k is *encoded* by a CPI path $\mathcal{P}$ if and only if $V_h(\mathcal{P}) \subseteq C_k \subseteq V_h(\mathcal{P}) \cup V_p(\mathcal{P})$; it is *covered* by $\mathcal{P}$ if and only if $C_k \subseteq V_h(\mathcal{P}) \cup V_p(\mathcal{P})$.

Example 3.2. For SCT as shown in Figure 3. $V_p(\mathcal{P}_1) = \{v_4, v_5\}$ and $V_h(\mathcal{P}_1) = \{v_1, v_6\}$. Thus, the number of 3-cliques on $\mathcal{P}_1$ is equal to $\binom{2}{3-2} = 2$. $V_p(\mathcal{P}_2) = \{v_2, v_3, v_4, v_5\}$ and $V_h(\mathcal{P}_2) = \{v_1\}$. Thus, the number of 3-cliques on $\mathcal{P}_2$ is equal to $\binom{4}{3-1} = 6$.

Inadequacies in SCT Structures. SCT is designed solely for clique counting on static graphs. For the key functions in our framework (Algorithm 2), SCT can support *s-CLIQUECOUNT* (i.e., initializing the support) with minor modifications. However, it lacks dynamic update capabilities and cannot be trivially extended to support *UPDATEINDEX* and *UPDATESUPPORT*, which are required in our representation layer to remove peeled r -cliques efficiently. This limitation motivates the design of a new index that can support all these operations, which is both non-trivial and challenging.

3.3 Clique Path Index

Building on the basic idea of SCT [19], we introduce a new data structure, the *Clique Path Index* (CPI), which is adapted to our counting-and-peeling framework and supports efficient updates. Unlike SCT, CPI represents



(a) An example of a CPI on Figure 1 (no pruning). (b) An example of a pruned CPI for $s = 4$ on Figure 3.

Fig. 4. Examples of CPI before and after pruning. Dashed nodes denote pivots, and solid nodes denote holds.

Algorithm 3: s -CLIQUECOUNT

Input: Clique Path Index $\mathcal{B}$, clique sizes $0 < r < s$

Output: $\text{count}(C_r)$ for every r -clique $C_r \subseteq G$

```

1 initialise  $\text{count}(C_r) \leftarrow 0$  for all  $r$ -cliques in  $\mathcal{B}$ ;
2 foreach clique path  $\mathcal{P}$  in  $\mathcal{B}$  do
3    $V_h \leftarrow \{v \in \mathcal{P} \mid v \text{ is hold}\};$ 
4    $V_p \leftarrow \{v \in \mathcal{P} \mid v \text{ is pivot}\};$ 
5   foreach  $r$ -set  $C_r \subseteq \mathcal{P}$  do
6      $b \leftarrow |C_r \cap V_p|$ 
7      $\text{count}(C_r) += \binom{|V_p| - b}{s - |V_h| - b};$ 
8 return  $\text{count}(C_r)$  for all  $r$ -cliques  $C_r$ ;
  
```

the search space as independent paths in a bipartite-like layout between vertices and paths, as illustrated in Figure 4a. Thanks to this bipartite-like storage structure, we can also efficiently retrieve the paths containing each vertex and the vertices contained in each path, facilitating the implementation of the `UPDATESUPPORT` function in our framework to accurately identify affected r -cliques. Updating a traditional tree is difficult, since shared prefixes cause cascaded changes and costly subtree reattachments; accordingly, CPI admits more efficient updates (details in the next section).

Example 3.3. Figure 4a shows an example of a CPI consisting of three paths and five vertices. Path 1 contains vertices v_1, v_2, v_3, v_4, v_5 , where v_1 is a hold vertex and the others are pivot vertices. Path 2 contains vertices v_1, v_4, v_5, v_6 , where v_1 and v_6 are hold vertices and the others are pivot vertices. Path 3 contains vertices v_2, v_3, v_4, v_5 , where v_2 is a hold vertex and the others are pivot vertices.

Based on Lemma 3.1, we derive Lemma 3.2, which enables the computation of the number of s -cliques for each r -clique without enumerating any s -cliques.

Lemma 3.2 (Local s -support of an r -clique). *Let $\mathcal{P}$ be a path in the CPI with pivot set V_p and hold set V_h ($h = |V_h|$, $p = |V_p|$), and fix integers $0 < r < s$. For any r -clique $C_r \subseteq \mathcal{P}$ set $b = |C_r \cap V_p|$, Then the number of distinct s -cliques on $\mathcal{P}$ that contain C_r equals $\#_s(C_r) = \binom{p-b}{s-h-b}$.*

PROOF. Every s -clique on the path is $V_h \cup Q$ with $Q \subseteq V_p$ and $|Q| = s - h$ (Lemma 3.1). Fix any r -clique C_r and let $b = |C_r \cap V_p|$. Since C_r already contributes those b pivots, an s -clique containing C_r is obtained precisely by choosing the remaining $s - h - b$ pivots from the $p - b$ unused pivots in V_p . The number of such choices is $\binom{p-b}{s-h-b}$, which is zero whenever the lower or upper bound is violated. $\square$

Based on Lemma 3.2, we derive Algorithm 3 to compute the number of s -cliques for each r -clique without explicit enumeration.

Path Pruning. Naively building the index would include information about all cliques in the graph. However, in (r, s) -nucleus decomposition, not all cliques are necessary—we are only interested in cliques that contains s -cliques. To accelerate index construction and reduce its size, we introduce the following pruning rule.

Algorithm 4: BUILD-CPI(G, s)**Input:** Undirected graph $G = (V, E)$, optional size cap s **Output:** Clique Path Index (CPI) of G

```

1 Compute a degeneracy ordering  $(v_1, \dots, v_n)$  of  $G$  Orient every edge from lower to higher rank to
  obtain  $N^+(\cdot)$ ;
2 for  $i \leftarrow 1$  to  $n$  do                                     // one root per vertex
3    $V_{Hold} \leftarrow \{v_i\}$   $P \leftarrow N^+(v_i)$   $V_{Pivot} \leftarrow \emptyset$ 
4    $\text{EXPANDPATH}(P, V_{Hold}, V_{Pivot})$ ;
5 Function  $\text{ExpandPath}(P, V_{Hold}, V_{Pivot})$ :
6   if  $|V_{Hold}| > s$  then return ;
7   if  $P = \emptyset$  then
8     if  $|V_{Hold}| + |V_{Pivot}| \geq s$  then
9       output path  $\{V_{Hold}, V_{Pivot}\}$ ;
10    return
11    $u_{piv} \leftarrow \arg \max_{x \in P} |P \cap N^+(x)|$  ;
12   for  $v \in P \setminus N(u_{piv})$  do                               // non-neighbors of pivot
13      $P \leftarrow P \setminus \{v\}$ 
14     if  $v = u_{piv}$  then
15        $\text{ExpandPath}(P \cap N(v), V_{Hold}, V_{Pivot} \cup \{v\})$ 
16     else
17        $\text{ExpandPath}(P \cap N(v), V_{Hold} \cup \{v\}, V_{Pivot})$ 

```

Lemma 3.3 (Feasibility of Path). *Let $\mathcal{P}$ be a path in the CPI and let $s \geq 1$ be the target clique size. Then $\mathcal{P}$ encodes at least one s -clique iff $|V_h(\mathcal{P})| \leq s \leq |V_h(\mathcal{P})| + |V_p(\mathcal{P})|$.*

PROOF. ($\Rightarrow$) According to Lemma 3.1, all s -cliques on $\mathcal{P}$ consist of vertices from $V_h(\mathcal{P})$ and $V_p(\mathcal{P})$. Therefore, if $s < |V_h(\mathcal{P})|$, it is impossible for any s -clique to include all hold vertices, and thus no s -clique exists. Similarly, if $s > |V_h(\mathcal{P})| + |V_p(\mathcal{P})|$, it is impossible for any s -clique to include all hold vertices and enough pivot vertices, and thus no s -clique exists. ($\Leftarrow$) Conversely, if s lies within this interval, we can construct at least one s -clique based on Lemma 3.1, ensuring the existence of at least one s -clique. $\square$

Example 3.4. Considering the CPI shown in Figure 4b, if we need to count 4-cliques, we can prune this CPI to retain only three paths, as illustrated in Figure 4b. Compared to the SCT (Figure 3), the pruned CPI has fewer paths, making it more efficient to store and traverse.

Index Construction. Algorithm 4 illustrates the construction process of CPI. Contrary to SCT, CPI outputs an entire path at once and applies Lemma 3.3 (Line 6 & 9) to prune paths during construction.

Since paths in the CPI can be pruned to retain only those that must contain s -cliques, we can also derive the following result.

Theorem 3.1 (Number of Clique Paths). *Let $\mathcal{B}$ be the CPI of a graph G , and let $\mathcal{P}_s = \{\mathcal{P} \mid |V_h(\mathcal{P})| \leq s \text{ and } |\mathcal{P}| \geq s\}$ be the set of CPI paths that remain after pruning for a given integer $s \geq 1$. Let $\mathcal{K}_s$ denote the set of all s -cliques in G . Then $|\mathcal{K}_s| \geq |\mathcal{P}_s|$. Moreover, $\sum_{P \in \mathcal{P}_s} |V_h(P)| \leq s |\mathcal{K}_s|$.*

PROOF. Let $\mathcal{P}$ be a path in the CPI $\mathcal{B}$. By Lemma 3.3, if $\mathcal{P}$ survives the pruning rule, it contains at least one s -clique. Thus, every path in $\mathcal{P}_s$ contains at least one s -clique. Therefore, the number of s -cliques is greater than or equal to the number of CPI paths in $\mathcal{P}_s$. By definition of $\mathcal{P}_s$, each $P \in \mathcal{P}_s$ satisfies $|V_h(P)| \leq s$. Thus $\sum_{P \in \mathcal{P}_s} |V_h(P)| \leq s |\mathcal{P}_s| \leq s |\mathcal{K}_s|$. $\square$

Remark. This theorem demonstrates a key advantage of our approach. The number of paths in the CPI cannot exceed the number of s -cliques, ensuring that storing the entire index is no more expensive than storing all s -cliques.

More importantly, since each path compactly represents multiple s -cliques, traversing and updating the CPI becomes substantially more efficient than operating on enumerated s -cliques. This property underpins the benefits of our representation layer, enabling efficient decomposition.

Complexity. The worst-case time to build the CPI is $O(n \cdot 3^{\alpha/3})$ [19], where α is the degeneracy of the graph, $n \cdot 3^{\alpha/3}$ is the maximum number of maximal cliques in a graph with n vertices. But in practice, due to path pruning, the actual construction time is significantly lower than this worst-case bound. Space complexity to store CPI is $O(\min(n \cdot 3^{\alpha/3}, |K_s|))$ (Theorem 3.1), where $|K_s|$ is the number of s -cliques in the graph.

From Counting to Decomposition. The CPI provides a powerful tool for computing the number of s -cliques associated with each r -clique while avoiding explicit s -clique enumeration. However, efficiently updating the CPI during computation and leveraging it to guide the decomposition process remain major technical challenges for performing nucleus core decomposition.

4 Differential Path Contribution Maintenance

The CPI introduced in the previous section resolves the *representation* challenge by encoding the s -clique space without explicit enumeration. The remaining challenge is algorithmic: after a batch of minimum-support r -cliques is peeled, how should the supports of the surviving r -cliques be updated *exactly* without reconstructing the affected local clique space from scratch? In this section, we expose a higher-level view of our method. We show that exact peeling on CPI can be understood as *differential path contribution maintenance*: each path contributes a local amount of support, and peeling only changes the contributions of paths incident to removed r -cliques.

Definition 4.1 (Active path state). Consider the peeling process at round t and let CPI_t denote the current CPI. For each active path $\mathcal{P} \in \text{CPI}_t$, let $\sigma_t(\mathcal{P})$ denote the local state maintained for $\mathcal{P}$. This state records exactly the information needed to determine which active s -cliques are still encoded by $\mathcal{P}$ and how they contribute to the active r -cliques covered by $\mathcal{P}$.

Definition 4.2 (Path contribution). Let $\mathcal{K}_r^{(t)}$ and $\mathcal{K}_s^{(t)}$ be the active r -cliques and s -cliques at round t . For an active r -clique $R \in \mathcal{K}_r^{(t)}$ and an active path $\mathcal{P} \in \text{CPI}_t$, define the *path contribution* of $\mathcal{P}$ to R as

$$\phi_t(R, \mathcal{P}) = |\{S \in \mathcal{K}_s^{(t)} \mid R \subseteq S, S \trianglelefteq \mathcal{P}\}|.$$

Intuitively, $\phi_t(R, \mathcal{P})$ counts how many currently valid s -cliques encoded by $\mathcal{P}$ contain R . The support of R is then obtained by summing these local contributions over all active paths.

Theorem 4.1 (Support decomposition by path contributions). *For every round t and every active r -clique $R \in \mathcal{K}_r^{(t)}$,*

$$\deg_{(s)}^{G_t}(R) = \sum_{\mathcal{P} \in \text{CPI}_t} \phi_t(R, \mathcal{P}).$$

PROOF. By Lemma 3.1, every active path encodes a well-defined set of active s -cliques, and each such s -clique contributes one unit of support to every contained r -clique. Moreover, the CPI path semantics ensure that each active s -clique is represented by exactly one path. Therefore, counting active s -cliques containing R is equivalent to summing the numbers contributed by all active paths. $\square$

Differential update view. Suppose that at round t the algorithm removes a batch $B_t \subseteq \mathcal{K}_r^{(t)}$ of minimum-support r -cliques. Only paths that contain removed cliques, or are created by splitting such paths, can change their contribution values. This yields a purely local update rule.

Definition 4.3 (Affected paths). Let $\Gamma_t(B_t)$ denote the set of paths that either: (i) contain some removed r -clique in B_t , or (ii) are newly created when such paths are updated or split.

Theorem 4.2 (Path locality of exact peeling). *Let R be any surviving r -clique after round t . If a path $\mathcal{P} \notin \Gamma_t(B_t)$, then*

$$\phi_t(R, \mathcal{P}) = \phi_{t+1}(R, \mathcal{P}).$$

Consequently,

$$\deg_{(s)}^{G_{t+1}}(R) = \deg_{(s)}^{G_t}(R) - \sum_{\mathcal{P} \in \Gamma_t(B_t)} (\phi_t(R, \mathcal{P}) - \phi_{t+1}(R, \mathcal{P})).$$

PROOF. If $\mathcal{P} \notin \Gamma_t(B_t)$, then neither the path nor the set of active s -cliques encoded by it changes during round t . Hence its contribution to every surviving r -clique remains unchanged. Summing only over affected paths gives the claimed differential form. $\square$

Theorem 4.2 is the main algorithmic message of this section: the global support update problem can be reduced to maintaining a small local state on affected paths. The remaining question is what state is sufficient in different regimes of r .

Theorem 4.3 (Regime-specific sufficient states). *Exact differential maintenance on CPI admits the following sufficient path states.*

- (1) $r=1$. It suffices to maintain whether a path is alive, the number of remaining pivots on the path, and the fixed number of pivots still needed to complete an s -clique. Hence every support delta is determined by constant-time binomial lookups.
- (2) $r=2$. It suffices to maintain the current hold and pivot sets of the path together with the local removal pattern that classifies the update into the three cases studied later. Dominant cases admit closed-form support deltas and analytical single-edge splits, avoiding general residual-graph search.
- (3) $r \geq 3$. It suffices to maintain the current hold/pivot partition, the conflict family induced by removed r -cliques, the residual pivot budget, and optional lazy cached clique information for the paths that survive. Dead paths can then be identified and handled without recursive splitting, while the remaining paths are updated exactly.

PROOF. Each item follows from the path counting semantics in Lemma 3.1 and from the later update rules derived for the corresponding regime. For $r=1$, the path contribution depends only on how many pivots remain, so counters are sufficient. For $r=2$, the contribution of an edge is determined by whether it is hold–hold, hold–pivot, or pivot–pivot, together with the removed pivot pattern. For $r \geq 3$, the effect of removals is captured by the conflict sets they induce on pivots and by whether the remaining pivot budget still allows a valid path to survive. $\square$

Corollary 4.1 (Exactness of DPCM-based peeling). *If every affected path applies the exact local update implied by its maintained state in Theorem 4.3, then the resulting peeling process computes the exact (r, s) -nucleus core number for every r -clique.*

PROOF. The proof is by induction on peeling rounds. Theorem 4.1 establishes the exact support decomposition at initialization. Theorem 4.2 shows that after each peeling step, only affected paths need to be updated. If these local updates are exact, then all surviving r -cliques receive exactly their new supports, so the next peeling choice matches the exact peeling process. $\square$

Why DPCM is beyond the conference version. The conference version introduced CPI as a compact *representation* of the s -clique space. It showed that this representation can replace explicit s -clique materialization. The journal-level extension is to separate representation from maintenance. CPI tells us *where* support is encoded. DPCM tells us *how* exact support changes during peeling. This separation gives a new algorithmic layer. This layer is not present in a purely counting-based view. We no longer maintain support by reconstructing affected clique neighborhoods. Instead, we update path-local sufficient states and recompute only the related differential contributions.

Implications for algorithm design. Once support maintenance is written in DPCM form, the design problem becomes clear. For each range of r , we need to find the smallest sufficient state $\sigma_t(\mathcal{P})$ that still supports exact local deltas. This explains why the three update mechanisms look different. They are still based on the same principle. For $r=1$, the sufficient state collapses to counters. For $r=2$, the sufficient state becomes an edge-contribution calculus with closed-form cases and analytical splits. For $r \geq 3$, the sufficient state is conflict-aware and supports dead-path pruning plus lazy residual maintenance. DPCM turns these three optimized algorithms into three exact instantiations of the same maintenance theory.

Table 1. Instantiation mapping of the unified peeling framework; the CPI/heap data structures are identical across regimes.

Regime	Peeled unit	s -clique support $\deg^{(s)}(\cdot)$	Affected units for local updates
General	r -clique	$\#\{K_s \mid C_r \subseteq K_s\}$	r -cliques on the updated CPI paths
$r = 2$	edge (2-clique)	$\#\{K_s \mid e \subseteq K_s\}$ (<i>triangle support</i> when $s=3$)	units co-occurring with e on the updated CPI paths
$r = 1$	vertex (1-clique)	$\#\{K_s \mid v \subseteq K_s\}$ (<i>deg</i> when $s=2$)	units co-occurring with v on the updated CPI paths

Instantiation map. The rest of the paper specializes this idea. Section 5 instantiates DPCM for the three main regimes. For $r=1$, the path state collapses to two counters and yields an immutable-tree implementation. For $r=2$, DPCM becomes a case-based edge contribution calculus with closed-form deltas and analytical path splits. For $r \geq 3$, DPCM becomes a conflict-aware lazy maintenance scheme. Expensive recursive splitting is used only when the maintained path state shows that a path is still alive.

5 Updating the Clique Path Index

In this section, we present how to efficiently update CPI during decomposition, specifically how to remove r -cliques (and their corresponding s -cliques) during the peeling process. This operation is one of the most challenging aspects of maintaining CPI, as it must preserve the path semantics defined in Lemma 3.1. Building on the differential path contribution maintenance view in Section 4, the goal here is to instantiate the exact local update operator for three regimes: general $r \geq 3$, the edge case $r=2$, and the vertex case $r=1$. We first introduce the general path-splitting principle for arbitrary r , and then show how it simplifies for $r = 2$ and $r = 1$ as implementation optimizations. Throughout peeling, we maintain the same CPI state across all r , only the instantiated update operator differs depending on whether the peeled object is a vertex ($r=1$), an edge ($r=2$), or a general r -clique. Table 1 summarizes how the same peeling/update pipeline instantiates for general r , $r = 2$, and $r = 1$.

We first define the semantics of removing an r -clique R from the CPI. After the update, no path in the CPI contains R . Namely, the operation removes exactly those cliques that include R , while preserving all others: every s -clique S with $R \not\subseteq S$ remains encoded by exactly one path in the CPI.

5.1 Instantiation III: Conflict-State DPCM for General r

In this subsection, we generalize the update strategy from vertices and edges to arbitrary r -cliques. Under Theorem 4.3, the sufficient state of an affected path now consists of its current hold/pivot partition together with the family of removed r -cliques that still conflict with the path. Thus the general regime becomes a *conflict-state DPCM*: each affected path maintains enough information to determine whether it survives, whether it must split, and how the surviving subpaths contribute to the remaining r -cliques.

Conflict Sets and Path Validity. Removing an r -clique R is a higher-order update. It simultaneously affects all s -cliques that contain R . We define the *conflict set* of R as its vertex set $V(R)$. After the update, no clique path may realize all vertices in $V(R)$ at the same time. Meanwhile, every remaining s -clique must still be encoded by exactly one path. On a path, vertices of R can appear as hold or pivot. Holds are mandatory, while pivots represent alternative choices. Therefore, updating the CPI reduces to removing exactly those realizations that include the whole conflict set. If needed, we split the affected path into a small number of new paths to restore validity and uniqueness (Lemma 3.1). Crucially, the number of new paths depends only on how many vertices of R act as pivots on that path. Building on this observation, Theorem 5.1 formalizes the general path-splitting rule for arbitrary r .

Theorem 5.1 (Clique Removal). *Let $\mathcal{P} = (V_p^{\mathcal{P}}, V_h^{\mathcal{P}})$ be a clique path, and let $C = (V_p^C, V_h^C)$ be an r -clique such that $V_p^C \subseteq V_p^{\mathcal{P}}$ and $V_h^C \subseteq V_h^{\mathcal{P}}$. Given an ordered set $V_p^C = \{v_0, \dots, v_{t-1}\}$, the updated path set $\mathcal{P}'$ is defined as:*

$$\mathcal{P}' = \left\{ (V_p^{\mathcal{P}} \setminus \{v_0, \dots, v_i\}, V_h^{\mathcal{P}} \cup \{v_0, \dots, v_{i-1}\}) \mid i = 0, \dots, t-1 \right\}.$$

PROOF. On a CPI path, holds are mandatory and pivots optional. For each i , promoting $v_0, \dots, v_{i-1}$ to holds and deleting v_i keeps the path valid and forbids any clique that contains all pivots of C , so no subpath

Algorithm 5: UPDATEINDEX

Input: clique path $\mathcal{P} = \{V_H, V_P\}$, set of r -cliques to remove $C_{\text{rm}} = \{C_0, \dots, C_{t-1}\}$, clique size s

Output: All subpaths induced after deleting C_{rm}

```

1 Build inverted index  $I(v) \leftarrow \{i \mid v \in C_i\}$  for all  $v \in \mathcal{P}$ 
2  $\text{cnt} \leftarrow$  array of size  $t$  initialized to  $r$ ;
3 PathSplit( $V_H, V_P$ );
4 Function PathSplit( $V_H, V_P$ )
5   if  $|V_H \cup V_P| < s$  or  $|V_H| > s$  then return;
6    $C_j \leftarrow$  the first  $r$ -clique in  $C_{\text{rm}}$  with  $\text{cnt}[j] = r$ ;
7   if no such  $C_j$  exists then
8     output path  $\{V_H, V_P\}$ ;
9     return
10  foreach  $v \in C_j \cap V_P$  do
11    foreach  $g \in I(v)$  do
12       $\text{cnt}[g] \leftarrow \text{cnt}[g] - 1$ 
13       $V_P \leftarrow V_P \setminus \{v\}$ ;
14      PathSplit( $V_H, V_P$ );
15       $V_H \leftarrow V_H \cup \{v\}$ ;
16      foreach  $g \in I(v)$  do
17         $\text{cnt}[g] \leftarrow \text{cnt}[g] + 1$ 

```

encodes C (soundness). For any surviving clique K , let i be the first index with $v_i \notin K$; then K contains the promoted prefix and excludes v_i , hence is encoded precisely on subpath i and on no other (completeness & uniqueness). $\square$

Example 5.1. Assume we have a clique path $\mathcal{P} = \langle v_1, v_2, v_3, v_4, v_5, v_6 \rangle$, where $V_P = \{v_1, v_2, v_3, v_6\}$ and $V_H = \{v_4, v_5\}$. If we have an r -clique $C = \{v_1, v_2, v_3, v_4\}$, then $V_P^C = \{v_1, v_2, v_3\}$ and $V_H^C = \{v_4\}$. According to Theorem 5.1, we can split $\mathcal{P}$ into three subpaths: $V_{p,0} = \{v_1, v_2, v_6\}$, $V_{h,0} = \{v_4, v_5\}$, $V_{p,1} = \{v_1, v_6\}$, $V_{h,1} = \{v_3, v_4, v_5\}$, $V_{p,2} = \{v_6\}$, $V_{h,2} = \{v_2, v_3, v_4, v_5\}$.

Remark. Theorem 5.1 allows a clique path $\mathcal{P}$ in the CPI to be split into multiple paths that exclude the r -clique C . This operation is crucial, as it implicitly removes r -cliques—without explicit enumeration—while preserving the correctness of the remaining structure.

Role in DPCM. Theorem 5.1 is the fundamental local state-transition operator for the general regime. It is exact but potentially expensive when many conflicting cliques hit the same path. The next subsection therefore adds a stronger journal-level refinement: dead-path-aware maintenance, which uses the maintained conflict state to certify that many affected paths can be discarded without recursive splitting.

Batched Updates. When multiple cliques are removed simultaneously, they often overlap, leading to redundant updates on shared vertices and paths. To mitigate this overhead, we aggregate overlap ping updates into vertex-centric batches. Specifically, we construct an *inverted index* that maps each vertex to the set of cliques containing it. Instead of processing cliques sequentially, we iterate over vertices in this index and update all affected clique paths per vertex. This vertex-grouped strategy effectively merges overlapping operations and reduce redundant updates.

Algorithm. Algorithm 5 maintains a counter cnt to track the number of vertices from each r -clique in C_{rm} that are still present on the current path. When cnt equals r , it indicates that the path violates the conflict clique and requires splitting. The splitting process (Function PathSplit) recursively handles each r -clique to be removed. First, in Line 6, we prune the path based on Lemma 3.3; if the current path cannot contain any cliques of size s , we return immediately. Then, in Line 7, we select the first r -clique C_j that needs to be removed. If no such r -clique exists, it indicates that the current path does not require any deletion operations,

so we report the current path as a subpath (Line 8). Otherwise, we iterate through each vertex v in the r -clique C_j , remove it from the path, and decrement the cnt of all r -cliques containing v by 1, as one vertex has been removed from these cliques. In Lines 15-17, we promote all pivot nodes before v to hold nodes according to Theorem 5.1, remove v from the path, and then recursively call the function `PathSplit` to process the updated path. Finally, in lines 19-20, we remove v from the path and continue processing the next vertex. According to Theorem 5.1, we can ensure that after each path split, the resulting subpaths do not contain the removed r -clique, and all size- s cliques that do not contain the removed r -clique are uniquely represented.

Correctness. We prove the correctness of Algorithm 5 in terms of soundness, completeness, and uniqueness. (*Soundness*) The algorithm maintains a counter array cnt to track how many vertices of each removed r -clique remain on the current path. A path is emitted only when all cnt values are strictly less than r , ensuring that the new path encodes no r -cliques marked for removal. (*Completeness & Uniqueness*) When removing vertices of an r -clique iteratively, at each deletion step, all remaining cliques not containing the removed vertex are re-encoded uniquely in new paths (by Theorem 5.1). Since every vertex is removed exactly once, all valid combinations of remaining vertices are exhaustively enumerated and retained. Therefore, every surviving clique is preserved and encoded by exactly one path in the updated CPI, ensuring both completeness and uniqueness.

Implementation. The size of the residual graph is bounded by the maximum clique size of the input graph, which is typically small for real-world datasets (Table 3). This allows us to represent the residual graph as an adjacency matrix and employ *bitsets* to encode vertex sets on paths, enabling highly efficient set operations (AND, OR, XOR, COUNT) through word-level parallelism [52].

Complexity. Let a clique path $\mathcal{P} = \{V_p(\mathcal{P}), V_h(\mathcal{P})\}$. The worst-case time complexity of Algorithm 5 is $O(\prod_{C \in C_{rm}} |C \cap V_p(\mathcal{P})|)$, as it may recursively split for each clique in C_{rm} . However, in practice, due to s -pruning and overlap among conflict cliques, the actual number of splits and the running time are significantly lower than this theoretical bound.

5.2 Instantiation II: Edge-Contribution DPCM for $r = 2$

For $r=2$, DPCM specializes to an *edge-contribution calculus*. The sufficient state of an affected path consists of its current hold/pivot partition together with the local removal pattern on that path. This state induces a small exact case split, and the dominant cases admit closed-form contribution updates rather than general residual-graph reconstruction. We start from the structural path-state transition in Theorem 5.2.

Theorem 5.2 (Edge Removal). *Let $\mathcal{P} = (V_p(\mathcal{P}), V_h(\mathcal{P}))$ be a clique path in the CPI of G , and let $e = \{u, v\}$ with $u, v \in V(\mathcal{P})$. For any removed e from G , the updated $\mathcal{P}'$ satisfies:*

- (1) (*hold-hold*) If $u, v \in V_h(\mathcal{P})$, then $\mathcal{P}' = \emptyset$;
- (2) (*hold-pivot*) If $u \in V_h(\mathcal{P})$ and $v \in V_p(\mathcal{P})$ (or symmetrically), then $\mathcal{P}' = \{(V_p(\mathcal{P}) \setminus \{v\}, V_h(\mathcal{P}))\}$;
- (3) (*pivot-pivot*) If $u, v \in V_p(\mathcal{P})$, then

$$\mathcal{P}' = \{(V_p(\mathcal{P}) \setminus \{v\}, V_h(\mathcal{P})), (V_p(\mathcal{P}) \setminus \{u\}, V_h(\mathcal{P}) \cup \{v\})\}.$$

PROOF. We prove the theorem separately for three distinct cases. (1) If e is a hold-hold edge, every encoded clique would require both endpoints, which becomes impossible after deleting e ; hence, the path is invalid. (2) If e is a hold-pivot edge, all encoded cliques must retain the hold endpoint. Thus, we update the path with respect to the pivot endpoint. (3) If e is a pivot-pivot edge, we split the path into two: (a) $(V_p(\mathcal{P}) \setminus \{v\}, V_h(\mathcal{P}))$, which removes all cliques encoded by the path that contain v ; (b) $(V_p(\mathcal{P}) \setminus \{u\}, V_h(\mathcal{P}) \cup \{v\})$, which restores all cliques containing v but not u . $\square$

DPCM Case Partition. Theorem 5.2 yields the exact $r=2$ case split required by Theorem 4.3. The hold-hold case kills the path immediately. The hold-pivot and pivot-only removal cases preserve a single path and admit direct support updates. The pivot-pivot case is the only branching case, and even there the dominant subcases can still be handled analytically.

Theorem 5.3 (Closed-Form Support Delta for Pivot Removal). *Let $\mathcal{P} = (V_p, V_h)$ be an active path with $P = |V_p|$, $K = |V_h|$, and $np = s - K$. Suppose a peeling round removes d pivots from $\mathcal{P}$ while keeping all holds alive and*

leaving the path feasible, i.e., $np \leq P - d$. Then the support change of each surviving edge e contributed by $\mathcal{P}$ depends only on its type:

$$\Delta_{\mathcal{P}}(e) = \begin{cases} \binom{P}{np} - \binom{P-d}{np}, & e \subseteq V_h, \\ \binom{P-1}{np-1} - \binom{P-d-1}{np-1}, & |e \cap V_h| = 1, \\ \binom{P-2}{np-2} - \binom{P-d-2}{np-2}, & e \subseteq V_p \text{ and both pivots survive.} \end{cases}$$

For any edge incident to a removed pivot, the full original contribution of $\mathcal{P}$ is subtracted.

PROOF. By Lemma 3.1, the contribution of an edge on $\mathcal{P}$ equals the number of ways to choose the remaining pivots needed to form a size- s clique. For a hold-hold edge, no pivot is fixed; for a hold-pivot edge, one pivot is fixed; for a surviving pivot-pivot edge, two pivots are fixed. The contribution in each case is therefore a single binomial term before and after the removal of d pivots, and the delta follows by subtraction. $\square$

Theorem 5.4 (Analytical Single-Edge Split). *Let $\mathcal{P} = (V_p, V_h)$ be an active path and let $\{u, v\} \subseteq V_p$ be the only removed pivot-pivot edge on $\mathcal{P}$. Then the surviving clique space on $\mathcal{P}$ is represented exactly by the two subpaths*

$$\mathcal{P}_1 = (V_p \setminus \{v\}, V_h), \quad \mathcal{P}_2 = (V_p \setminus \{u, v\}, V_h \cup \{v\}).$$

Moreover, the support contribution on each subpath is obtained directly from Lemma 3.1, so no residual-graph search is needed.

PROOF. Every surviving size- s clique encoded by $\mathcal{P}$ either excludes v or includes v and excludes u . The first family is encoded exactly by $\mathcal{P}_1$. The second family is encoded exactly by $\mathcal{P}_2$, where v is promoted to a hold because every clique in that family must contain it. These two families are disjoint and exhaustive over all cliques on $\mathcal{P}$ that avoid the removed edge. $\square$

Batched Updates. Theorem 5.2 specifies how a single edge deletion updates one clique path. In peeling, many edges are removed together. Processing them one by one revisits the same paths repeatedly. This is most expensive for pivot-pivot deletions, where each deletion may split a path and trigger cascading updates.

We use a *path-local* batched DPCM strategy. For each affected clique path, we first merge all removed edges that touch this path. We then handle hold-hold deletions by killing the path, and hold-pivot/ pivot-only deletions through the closed-form update of Theorem 5.3. If exactly one pivot-pivot edge remains, we apply Theorem 5.4. Only the residual multi-edge pivot-pivot case falls back to a residual graph on $V(\mathcal{P})$ after removing these edges. Finally, we reconstruct the resulting clique paths from the residual graph in a single pass. The residual graph is small (bounded by the maximum clique size), so the reconstruction is efficient.

Key Idea. We first handle the simple cases (hold-hold removals delete the entire path, and hold-pivot removals exclude the pivot), before addressing the more complex scenario of removing multiple pivot-pivot paths simultaneously. For pivot-pivot paths, the core idea is to first construct a *residual graph* from vertices in $V(\mathcal{P})$, retaining only the edges that remain valid within this subgraph. We then reconstruct new clique paths from the residual graph. Since the residual graph is small (no larger than the maximum clique in the graph), this reconstruction can be performed efficiently, avoiding redundant computation.

Algorithm. The algorithm (Algorithm 6) takes as input a clique path $\mathcal{P}$, a set E_{rm} of edges to remove, and the target clique size s (for pruning). It first constructs the residual graph H by deleting all edges in E_{rm} from the complete graph induced by $V(\mathcal{P})$. Next, it identifies the set of *fixed* vertices V_{fixed} in H that are not incident to any removed edge, and determines the subset of pivots among these fixed vertices. The sets R and P are then initialized as $R = V_{\text{fixed}}$ and $P = V(\mathcal{P}) \setminus V_{\text{fixed}}$, respectively. Finally, a recursive Bron-Kerbosch-like search function, `PathUpdate`, is invoked to enumerate all valid subpaths from the residual graph. In each recursive call, a pivot vertex connected to the largest number of vertices in P is selected [7], and the search expands along its connected candidates to generate the cliques that define the updated path(s).

Correctness. Cases (1)-(2) follow directly from the hold/pivot rules, and case (3) holds because the residual graph deletes exactly the removed pivot-pivot edges while preserving all remaining edges; Bron-Kerbosch enumeration on this residual graph yields all surviving cliques uniquely.

Complexity. Let a clique path $\mathcal{P} = \{V_p(\mathcal{P}), V_h(\mathcal{P})\}$. Let $L = |V_h(\mathcal{P})| + |V_p(\mathcal{P})|$ and let ω_H be the maximum clique size in the residual graph H on $V(\mathcal{P})$. If a hold-hold case is triggered, we return in $O(1)$ time. If the

Algorithm 6: UPDATEINDEX ($r = 2$)**Input:** clique path $\mathcal{P} = \{V_H, V_P\}$, remove-edge set E_{rm} , $s > 0$ **Output:** All subpaths induced after deleting E_{rm}

```

1 if  $\exists e \in E_{rm} : e \subseteq V_h(\mathcal{P})$  then return;
2 foreach  $e \in E_{rm} : |e \cap V_h(\mathcal{P})| = 1$  do
3    $\text{remove } (e \cap V_p(\mathcal{P})) \text{ from } \mathcal{P}$ 
4    $\text{remove } e \text{ from } E_{rm}$ 
5  $H \leftarrow$  a residual graph on  $V(\mathcal{P})$  with edges in  $E_{rm}$  removed
6  $v_{fixed} \leftarrow \{v \in V(H) \mid v \text{ is not incident to any edge of } E_{rm}\}$ 
7  $Piv \leftarrow \{v \in v_{fixed} \mid v \text{ is a pivot on } \mathcal{P}\}$ 
8  $R \leftarrow v_{fixed}, \quad P \leftarrow V(H) \setminus v_{fixed}$ 
9 PathUpdate( $R, P, Piv$ )
10 Function PathUpdate( $R, P, Piv$ ):
11   if  $|R \setminus Piv| > s$  then return;
12   if  $P = \emptyset$  then
13     if  $|R| \leq s$  then output path  $\{R, Piv\};$ 
14     return
15    $u_{piv} \leftarrow \arg \max_{x \in P} |P \cap N_H(x)|;$ 
16   foreach  $v \in P \setminus N_H(u_{piv})$  do
17      $R' \leftarrow R \cup \{v\}; \quad P' \leftarrow P \cap N_H(v);$ 
18     if  $v = u_{piv}$  then  $Piv' \leftarrow Piv \cup \{v\};$ 
19     else  $Piv' \leftarrow Piv;$ 
20     PathUpdate( $R', P', Piv'$ );
21    $P \leftarrow P \setminus \{v\};$ 

```

update falls into the closed-form cases of Theorem 5.3 or the analytical split of Theorem 5.4, the update costs only $O(L)$ time. Only the residual multi-edge pivot-pivot case requires the local Bron-Kerbosch-like search on H , which costs $O(3^{L/3})$ time and $O(L^2)$ space in the worst case.

5.3 Instantiation I: Counter-State DPCM for $r = 1$

Under Theorem 4.3, the sufficient path state for $r=1$ collapses to a tiny counter-based object. For each active path $\mathcal{P}$, we only need to know whether it is alive, the number of remaining pivots on the path, and the fixed number of pivots still required to form a size- s clique. Hence the $r=1$ regime becomes an immutable-path instance of DPCM: the path transition is simple, and the support change of every surviving vertex is given by a closed-form difference. The underlying path-state transition is described in Theorem 5.5.

Theorem 5.5 (Vertex Removal). *Let $\mathcal{P}$ be a clique path in the CPI of a graph G , with hold and pivot sets denoted by $V_h(\mathcal{P})$ and $V_p(\mathcal{P})$, respectively. For any removed vertex $v \in \mathcal{P}$, the updated $\mathcal{P}'$ satisfies:*

- (1) if $v \in V_h(\mathcal{P})$, $\mathcal{P}' = \emptyset$;
- (2) if $v \in V_p(\mathcal{P})$, $\mathcal{P}' = \{(V_p(\mathcal{P}) \setminus \{v\}, V_h(\mathcal{P}))\}$.

PROOF. By Lemma 3.1, every k -clique encoded by a path $\mathcal{P}$ must contain all hold vertices on that path, whereas pivot vertices are optional. If the removed vertex v lies in the hold set $V_h(\mathcal{P})$, no encoded k -clique can survive, so the whole path vanishes. If $v \in V_p(\mathcal{P})$, only those cliques that actually include v disappear; the cliques formed by the unchanged hold set and the remaining pivots are still uniquely represented by the shortened path, so we remove v and keep the rest of $\mathcal{P}$ intact. $\square$

Batched Updates. In each iteration of the peeling process, many r -cliques are removed. To avoid redundant computation, we adopt a batched strategy that merges repeated updates. For vertex removals, the algorithm

can be easily extended to handle batches. Given a set of vertices to be removed V_{rm} and a path $\mathcal{P}$ containing any of them, we update $\mathcal{P}$ as follows:

- (1) $\mathcal{P}' = \emptyset$ if $|V_h(\mathcal{P}) \cap V_{rm}| > 0$;
- (2) $\mathcal{P}' = \{(V_p(\mathcal{P}) \setminus V_{rm}, V_h(\mathcal{P}))\}$ if $|V_h(\mathcal{P}) \cap V_{rm}| = 0$.

We can easily update $\mathcal{P}$ according to the above rules.

Closed-Form Contribution Delta. For $r=1$, the local support contribution of a path depends only on how many pivots remain after a peeling round. This yields a fully analytical DPCM update rule.

Theorem 5.6 (Closed-Form Support Delta for $r = 1$). *Let $\mathcal{P} = (V_p, V_h)$ be an active path with $rp = |V_p|$ remaining pivots and $np = s - |V_h|$ pivots still needed to form a size- s clique. Suppose that a peeling round removes d pivots from $\mathcal{P}$ and no hold vertex on $\mathcal{P}$ is removed. If the path survives, i.e., $np \leq rp - d$, then the support change of each surviving vertex u contributed by $\mathcal{P}$ is*

$$\Delta_{\mathcal{P}}(u) = \begin{cases} \binom{rp}{np} - \binom{rp-d}{np}, & u \in V_h, \\ \binom{rp-1}{np-1} - \binom{rp-d-1}{np-1}, & u \in V_p. \end{cases}$$

If a hold is removed or $np > rp - d$, then the path dies and its entire contribution is subtracted.

PROOF. By Lemma 3.1, a hold vertex belongs to every size- s clique encoded by $\mathcal{P}$, so its path contribution equals the number of ways to choose the required pivots, namely $\binom{rp}{np}$ before the update and $\binom{rp-d}{np}$ after it. For a surviving pivot vertex, one pivot is fixed by the vertex itself, so the corresponding counts become $\binom{rp-1}{np-1}$ and $\binom{rp-d-1}{np-1}$. The support delta follows by subtraction. $\square$

Implication for DPCM. Theorem 5.6 shows that the $r=1$ instantiation needs neither path splitting nor local clique reconstruction. All exact support updates are reduced to constant-time binomial lookups on the maintained path counters.

Complexity. Let a clique path $\mathcal{P} = \{V_p(\mathcal{P}), V_h(\mathcal{P})\}$. For a single path update, the running time of removing a node from CPI is $O(|V_h(\mathcal{P})| + |V_p(\mathcal{P})|)$ and the extra space is $O(1)$. By Theorem 5.6, the differential support update on the path is fully analytical, so the overall update cost stays linear in the path length.

5.4 Dead-Path-Aware DPCM for General r

The previous subsections instantiate DPCM for $r=1$, $r=2$, and general r . For $r=1$ and $r=2$, the dominant updates are already reduced to counter-only or closed-form operators. The remaining bottleneck appears in the general regime, where conflict-state DPCM may still invoke recursive path splitting. In this subsection, we introduce a stronger journal-level refinement for that regime: *dead-path-aware* maintenance. The idea is to use the maintained conflict state to certify that many affected paths cannot possibly encode any surviving size- s clique, allowing us to discard them before recursion.

Key Idea: Budgeted Hitting Set. When multiple r -cliques are removed from a single clique path $\mathcal{P}$, Algorithm 5 must determine which subpaths survive. Before invoking the recursive search, we apply a polynomial-time test that detects *dead paths*—paths that cannot produce any valid subpath of size $\geq s$. If the test succeeds, we skip the exponential splitting entirely and subtract the remaining support in $O(\binom{|\mathcal{P}|}{r})$ time.

Definition 5.1 (Conflict Family). Given a set of removed r -cliques $C_{rm} = \{C_0, \dots, C_{m-1}\}$ on path $\mathcal{P}$, the *conflict family* is $\mathcal{F} = \{F_0, \dots, F_{m-1}\}$ where $F_i = C_i \cap V_p(\mathcal{P})$ is the set of pivot members of C_i . The *budget* is $B = |V_p(\mathcal{P})| - (s - |V_h(\mathcal{P})|)$: the maximum number of pivots that may be removed.

Any valid subpath must exclude at least one pivot from each F_i while retaining at least $s - |V_h|$ pivots. This is precisely a *budgeted hitting set* problem. Computing the exact minimum hitting set is NP-hard, but the following polynomial-time conditions suffice for most practical instances.

Lemma 5.1 (Forced Vertex). *If a vertex v belongs to every r -clique on $\mathcal{P}$ (i.e., its conflict degree equals $\binom{|\mathcal{P}|-1}{r-1}$), then v must be removed in every feasible solution. If v is a hold, the path is dead.*

Lemma 5.2 (Singleton Propagation). *If $|F_i| = 1$, say $F_i = \{p\}$, then p must be removed. Removing p may create new singletons; we propagate iteratively until no singleton remains. If at any point $|V_p| + |V_h| < s$, the path is dead.*

Algorithm 7: PRUNEDUPDATEINDEX**Input:** clique path $\mathcal{P} = \{V_H, V_P\}$, r -cliques to remove C_{rm} , clique size s **Output:** All subpaths after deletion, or $\emptyset$ if dead

```

1  $F_i \leftarrow C_i \cap V_P$  for each  $C_i \in C_{rm}$ ; // conflict sets
2  $B \leftarrow |V_P| - (s - |V_H|)$ ; // removal budget
   // Forced vertex (Lemma 5.1)
3 foreach  $v$  with conflict degree  $\geq \binom{|\mathcal{P}|-1}{r-1}$  do
4   if  $v \in V_H$  then return  $\emptyset$ ;
5    $V_P \leftarrow V_P \setminus \{v\}$ ; resolve all  $F_i \ni v$ ;
   // Singleton propagation (Lemma 5.2)
6 repeat
7   foreach active  $F_i$  with  $|F_i| = 1$ , say  $\{p\}$  do
8      $V_P \leftarrow V_P \setminus \{p\}$ ; resolve all  $F_j \ni p$ ;
9     if  $|V_H| + |V_P| < s$  then return  $\emptyset$ ;
10 until no change;
    // Component packing LB (Lemmas 5.3, 5.4)
11 Build conflict graph  $G_c$ ; find components  $C_1, \dots, C_k$ ;
12  $LB \leftarrow \sum_{j=1}^k \text{GREEDYPACKING}(\{F_i \mid F_i \subseteq C_j\})$ ;
13 if  $LB > B$  then return  $\emptyset$ ;
14 return PathSplit( $V_H, V_P$ ) with reduced  $\mathcal{F}$ ; // Alg. 5
```

Lemma 5.3 (Disjoint Packing Lower Bound). *If there exist t pairwise disjoint sets in $\mathcal{F}$, then any feasible solution must remove $\geq t$ pivots. If $t > B$, the path is dead.*

Lemma 5.4 (Component Decomposition). *Let G_c be the graph on V_P where two pivots are adjacent iff they co-occur in some F_i . If G_c has connected components $C_1, \dots, C_k$, then the minimum hitting set decomposes as $\tau(\mathcal{F}) = \sum_j \tau(\mathcal{F}[C_j])$. Computing the packing lower bound per component and summing yields a bound at least as tight as the global one.*

These conditions are applied as a preprocessing step before Algorithm 5. Algorithm 7 summarizes the procedure.

Effectiveness. Our experiments show that the above tests prune the vast majority of affected paths on representative datasets. At $s = 20$, up to 97% of paths are detected as dead, reducing the number of paths that enter the exponential splitting (Algorithm 5) by up to 47 $\times$. This is the primary reason our method completes on parameter ranges where the baseline times out.

5.5 Complexity.

Table 2 summarizes time/space. Let K_r and K_s be the sets of r - and s -cliques, and let CPI be the clique-path index. The total time consists of (i) build/initial counting, (ii) CPI maintenance during peeling, and (iii) support/heap maintenance.

(i) *Build.* Building CPI and initializing supports takes $O(|\text{CPI}| + |K_r|)$ (Section 3.3).

(ii) *CPI maintenance.* Let $T_{\text{UpdateIndex}}$ denote the total cost of maintaining CPI over all peeling steps; it specializes for general r , $r=2$, and $r=1$ as analyzed in the previous sections.

Regime-sensitive maintenance cost. The journal-level role of DPCM is precisely to reduce $T_{\text{UpdateIndex}}$ by replacing reconstructive updates with state-local differential updates. For $r=1$, Theorem 5.6 shows that path maintenance is counter-only, so each affected path is processed in time linear in the path length. For $r=2$, Theorems 5.3 and 5.4 show that the dominant affected paths are handled in $O(|\mathcal{P}|)$ time, and only the residual multi-edge pivot case falls back to local residual-graph search. For general r , Theorem 5.1 gives the exact conflict-state transition, while the dead-path-aware refinement in Section 21 removes most paths before

Table 2. Complexity summary. κ is arboricity, $ct_r(v) = |\{C_r \in K_r : v \in C_r\}|$, and l is the number of peeling rounds.

Method	Time = <i>Build/init</i> + <i>Support updates</i> + <i>Peel-min</i>
Enum. [44]	$O(m\kappa^{r-2}) + O(\sum_{v \in V} ct_r(v) \deg(v)^{s-r}) + O(l \log K_r)$
CPI-based	$O(CPI + K_r) + (T_{\text{UpdateIndex}} + O(\sum_{P \in CPI} P _r \log K_r)) + O(l \log K_r)$
Space	Enum.: $O(K_r)$; CPI: $O(CPI + K_r)$

Algorithm 8: UPDATESUPPORT(Hierarchy)

Input: Remove r -clique se $R_{\min}$, minimum heap $H_{\min}$, Clique Path Index $\mathcal{B}$, s -support map *support*

Output: Updated *support* and union-find structure

```

1 foreach clique path  $\mathcal{P}$  containing  $rc \in R_{\min}$  do
2   Update support( $c$ ) and  $H_{\min}$  for all  $s$ -clique  $c \in \mathcal{P}$ 
3   UniteLevel( $rc, c, \text{currentK}$ ) for all  $s$ -clique  $c \in \mathcal{P}$ 
4 Function UniteLevel( $a, b, \text{currentK}$ ):
5   for  $k \leftarrow \text{currentK}$  to 0 do
6     if not codeDisjointSets[ $k$ ].unite( $a, b$ ) then
7       return;

```

recursive splitting. Thus the same global complexity form hides three qualitatively different maintenance primitives, which is exactly the regime-specific benefit exposed by DPCM.

(iii) *Supports/heap*. Let l be the number of peeling rounds (distinct minimum-support values), and let $|P|_r$ be the number of affected r -cliques on path P . Using a bucketed min-heap over K_r , the total time for extracting minima and key updates is $O((l + \sum_{P \in CPI} |P|_r) \log |K_r|)$.

Worst case. By theorem 3.1, $|CPI| \leq |K_s|$ and $\sum_{P \in CPI} |P|_r \leq s|K_s|$ always holds. But If G is a disjoint union of s -cliques, then $|CPI| = |K_s|$ and our method degenerates to enumerating all s -cliques.

6 Hierarchy Construction

Thanks to our framework, we can extend the nucleus decomposition to efficiently construct the hierarchy of nuclei. The hierarchy is represented as a tree structure, where each node corresponds to a nucleus and edges denote containment relationships between nuclei. The root of the tree represents the entire graph, while the leaves correspond to the smallest, most cohesive nuclei. We formally define hierarchy construction in the context of nucleus decomposition.

Definition 6.1 (s -Connectivity). Given integers $0 < r < s$, two r -cliques R and R' in a graph (or subgraph) H are said to be s -connected if there exists a sequence of r -cliques $R = R_1, R_2, \dots, R_t = R'$, referred to as an s -clique path, such that for each $i \in \{1, \dots, t-1\}$ there exists an induced s -clique $S_i \subseteq V(H)$ satisfying $R_i \cup R_{i+1} \subseteq S_i$.

Definition 6.2 (s -Connected Component). For integers $0 < r < s$, the s -connected component of an r -clique R is the induced subgraph

$$\text{Comp}_s(R) = G \left[\bigcup_{R' : R' \text{ is } s\text{-connected to } R} R' \right].$$

It is the maximal subgraph where all r -cliques are mutually s -connected, and such components partition all r -cliques in G .

Hierarchy can be constructed by identifying containment relationships among nuclei. Specifically, an (r, s) -nucleus N is said to *contain* another (r', s') -nucleus N' if every r' -clique in N' is also an r -clique in N , and $s' \geq s$. This containment relationship is represented as a directed edge from N to N' in the hierarchy tree. The CPI can be used to efficiently determine whether two r -cliques are s -connected.

Table 3. Statistics of Datasets.

Dataset	n	m	Avg deg.	Max deg.	#Triangles	Max Clique
com-dblp(DBLP)	317 081	1 049 867	6.6	343	2 224 385	114
web-Stanford(STF)	281 903	1 992 636	14.1	38 625	11 329 473	61
com-youtube(YT)	1 134 890	2 987 624	5.3	28 754	3 056 386	17
web-Google(GO)	875 713	4 322 051	9.9	6 332	13 391 903	44
web-it-2004(WI)	509 338	7 178 413	28.2	469	338 884 212	432
soc-pokec(PO)	1 632 803	22 301 964	27.3	14 854	32 557 458	29

Theorem 6.1 (Clique Connectivity). *Given Two r -cliques C and C' and a clique path $\mathcal{P}$ of the CPI, if $C \subseteq \mathcal{P}$ and $C' \subseteq \mathcal{P}$, then C and C' are s -connected.*

PROOF. If $C \subseteq \mathcal{P}$ and $C' \subseteq \mathcal{P}$, then there exists a sequence of r -cliques $C = C^1, C^2, \dots, C^k = C'$ such that each consecutive pair C^i and C^{i+1} are s -connected. This is because each r -clique in the sequence shares an s -clique with the next one, as they are all part of the same clique path $\mathcal{P}$. Therefore, by the definition of s -connectedness, C and C' are s -connected. $\square$

Based on the above theorem, we can use a multi-level disjoint set union (DSU) data structure to identify all (r, s) -nuclei. Specifically, we maintain a multi-level DSU where each level corresponds to a k -core, and within each level, we maintain the connected components of r -cliques. As we perform the core decomposition, we dynamically update these connected components. The pseudocode is shown in Algorithm 8.

Algorithm. The procedure in Algorithm 8 follows the base pipeline; we keep a stack of DSUs (one per level) to maintain s -components. For each batch $S_{\min}$ assigned level k , `Uni teLevel` applies unions on affected CPI paths and replicates each successful union to all levels at most k .

Correctness. A k -(r, s)-nucleus is always a subgraph of a $(k - 1)$ -(r, s)-nucleus [41]. This implies that if two r -cliques are connected at level k , they must also remain connected at level $k - 1$. Therefore, if two r -cliques are found to be connected during peeling by Theorem 6.1, they remain connected at all lower levels. This property defines the behavior of the `Uni teLevel` function. For the correctness of the early return (Line 6): if it were incorrect, there would exist a scenario in which two r -cliques are connected in the k -(r, s)-nucleus but not correctly connected in the $(k - 1)$ -(r, s)-nucleus. Such a situation is impossible because, during the peeling process, $c_{\min}$ is monotonically increasing, and each union operation is consistently propagated to all lower levels.

Complexity. The complexity is identical to that of nucleus decomposition.

7 Experimental Evaluation

Hardware. All algorithms are implemented in C++ and compiled using the g++ compiler with -O3 optimization. Experiments were conducted on a machine equipped with dual Intel(R) Xeon(R) Gold 6342 2.80GHz CPUs, 500GB of RAM, and running Ubuntu 22.

Datasets. We evaluate our approach on six real-world datasets from diverse domains, including Co-authorship Networks: *com-dblp* (cblp) [60]; Social Networks: *soc-pokec* (pokec) [49], *com-youtube* (youtube) [60]; and Web Graphs: *web-Google* (google) [25], *web-it-2004* (web-it) [36], *web-Stanford* (stanford) [25]. These datasets vary in size and structural characteristics, providing a comprehensive evaluation of our approach. Table 3 summarizes the key statistics of these datasets.

Algorithms. We compare our method with two state-of-the-art methods from the ARB line: the peeling framework ARB without hierarchy [44]¹ and its hierarchy construction extension [45]². and the local h -index framework ND from [40]³. For ND, in addition to its base implementation, we also evaluate its two optimized parallel variants available in the same repository: the peeling-based PND – peel and the optimized local h -index PND – Opt. ARB, ND and our method support hierarchy construction and core decomposition, while

¹<https://github.com/jeshi96/arb-nucleus-decomp>

²<https://github.com/jeshi96/arb-nucleus-hierarchy>

³<https://github.com/sariyuce/nucleus>

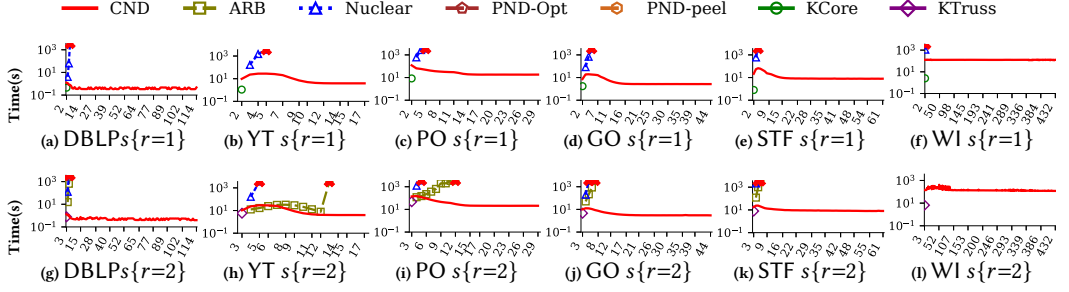


Fig. 5. Running time (seconds) across s without hierarchy for $r = 1$ (a-f) and $r = 2$ (g-l).

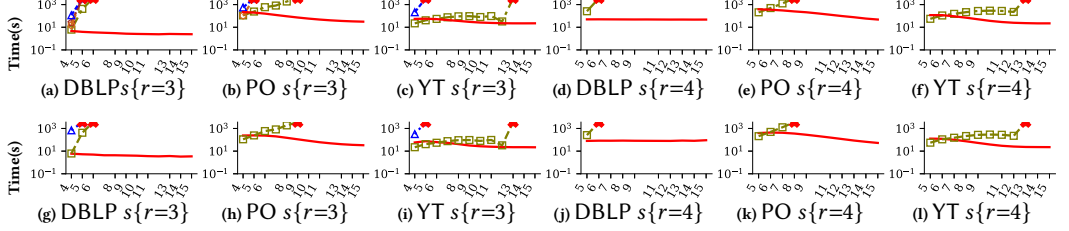


Fig. 6. Running time (seconds) across s for $r \in \{3, 4\}$ (a-f without hierarchy; g-l with hierarchy).

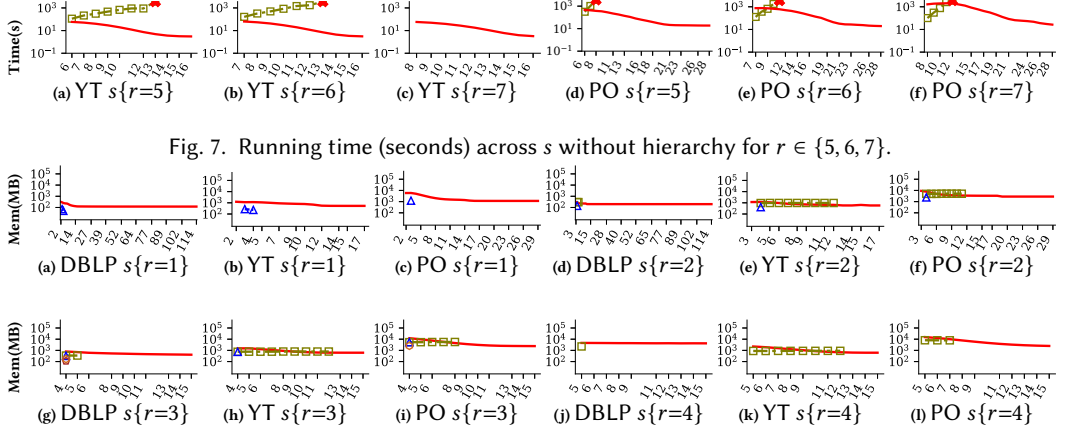


Fig. 7. Running time (seconds) across s without hierarchy for $r \in \{5, 6, 7\}$.

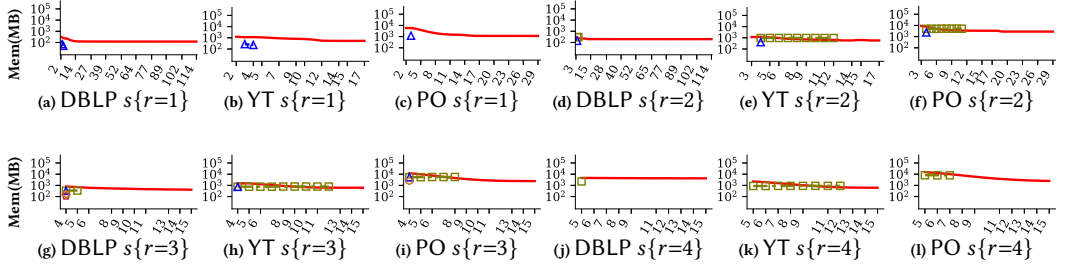


Fig. 8. Time and Memory usage comparison for $r \in \{1, 2, 3, 4\}$.

the PND – Opt and PND – peel focus on decomposition. Notably, ARB does not support $r = 1$ due to its algorithmic design. Regarding ND, its base implementation only supports the cases of $(r, s) = (1, 3)$, $(1, 4)$, $(2, 3)$, and $(3, 4)$. Furthermore, PND – peel and PND – Opt, are even more specialized and only support the case of $(r, s) = (3, 4)$. Our method is publicly available.⁴ Although these baselines differ in implementation details, they all follow the same exact peeling skeleton reviewed in Section 17 and rely on more reconstructive support-maintenance primitives. They differ in implementation. But they use the same exact peeling skeleton reviewed in Section 17. They also use more reconstructive support-maintenance primitives. Our end-to-end comparison therefore evaluates both layers of our method. It evaluates the counting/index layer provided by CPI. It also evaluates the DPCM update layer introduced in Section 4.

⁴<https://github.com/JamesZwq/nuclearSigmod/>

Metrics. Each experiment is executed three times, and we report the *average* wall-clock runtime. A timeout of **3600 seconds** (1 hour) is imposed; runs exceeding this limit are marked as *out of time* (OOT) (denoted by ✗ in figures) and excluded from the averages.

Evaluation Questions. Our experiments are designed to answer four questions. **Q1:** Does the combined CPI + DPCM framework consistently outperform prior exact peeling methods across different values of r and s ? **Q2:** Do the three DPCM instantiations yield the expected regime-specific benefits, namely counter-state updates for $r=1$, edge-contribution updates for $r=2$, and conflict-state pruning for $r \geq 3$? **Q3:** How much of the gain comes from avoiding reconstructive support maintenance, as opposed to merely changing the underlying representation? **Q4:** Does the same framework also preserve the efficiency advantage when hierarchy construction is included? Questions Q1–Q3 separate the two layers of our approach. Q1 studies the end-to-end effect of combining representation and maintenance. Q2 studies whether the regime-specific sufficient states predicted by DPCM appear in practice. Q3 separates the compression benefit of CPI from the update benefit of differential maintenance. Q4 checks whether the same advantages remain after we add connectivity maintenance for hierarchy construction.

Exp-1: Running Time ($r \leq 2$). We evaluate $r \in \{1, 2\}$ without hierarchy on six datasets, sweeping s from $3/4$ up to each dataset’s maximum clique size (Figure 5). These results answer Q1. They also answer the low-order part of Q2. CND shows little sensitivity to s : curves are essentially flat across datasets. This is the behavior predicted by counter-state and edge-contribution DPCM. Once CPI compactly represents the clique space, support updates do not need to reconstruct large affected local neighborhoods as s grows. In contrast, ARB and ND grow rapidly with s and often time out. Atypical is *com-youtube* at $r=2$ (max clique 17), where the gap narrows; even there, ARB times out at $s=13$ while CND finishes in about 30s. On *web-it-2004* ($r=2$), ARB times out for all s , whereas CND completes from $s=4$ onward (e.g., 240s at $s=4$) and handles the full range. On comparable settings, CND is about $6.3\times$ faster than ARB and about $18.0\times$ faster than ND. The largest gap is above two orders of magnitude. We further compare against specialized single-thread baselines for the two canonical special cases: k -core ($(r, s) = (1, 2)$) [3] and k -truss ($(r, s) = (2, 3)$) [55]. On *com-dblp*, CND runs in 1.21s for $(1, 2)$ vs. 0.45s for k -core, and 1.31s for $(2, 3)$ vs. 0.78s for k -truss. This result shows that our combined CPI+DPCM framework has only modest overhead on the special cases. It still keeps the advantage of avoiding explicit clique enumeration. It also remains effective as s increases.

Exp-2: Running Time ($r \in \{3, 4\}$). We evaluate $r \in \{3, 4\}$ on three datasets, with and without hierarchy (Figure 6).

Exp-2.1: Without Hierarchy. Figure 6 compares CND with ND and ARB. This part mainly answers the general-regime side of Q2. It also helps answer Q3. As with $r \leq 2$, CND is nearly insensitive to s , curves stay essentially flat, while ARB (and ND) grow rapidly with s and frequently time out. The key difference is that the gain here does not come from simple counter or edge states. It comes from conflict-state DPCM. Dead paths are pruned early. Only the surviving residual paths need further exact maintenance. On comparable settings, CND is about $2.8\times$ faster than ARB and about $4.6\times$ faster than the base ND implementation. The largest speedup reaches $106.7\times$ on *com-dblp* at $r=3, s=5$. We also evaluate the specialized algorithms PND – peel and PND – Opt for $(r, s) = (3, 4)$. On *com-dblp*, CND achieves $3.2\times$ and $5.5\times$ speedups, respectively. However, CND is less effective on graphs with small max cliques but large average degrees (e.g., *soc-pokec* and *web-Google*). In these datasets, for $(r, s) = (3, 4)$, PND – Opt, PND – peel, and ARB are all around $1\times$ faster than CND; however, as s increases (e.g., $(3, 5)$ and $(4, 5)$), CND quickly gains a strong advantage. In such cases, the enumeration cost for baselines is naturally low, while the dense connectivity increases the maintenance overhead of our index.

Exp-2.2: With Hierarchy. Figure 6.2 shows the hierarchical setting. This addresses Q4. The same trend appears in the hierarchical setting. CND stays stable as s increases, whereas the baselines often slow down or time out. On comparable settings, CND is about $2.6\times$ faster than ARB and about $18.4\times$ faster than ND. The largest speedup reaches $146.6\times$ on *com-dblp* at $r=3, s=4$.

Exp-3: Running Time ($r \in \{5, 6, 7\}$). Figure 7 reports the efficiency for higher-order nucleus decomposition on *soc-pokec* and *com-youtube*. As r increases, CND maintains a robust and relatively flat performance curve. In contrast, ARB exhibits a non-monotonic trend. On *soc-pokec*, the speedup of CND over ARB narrows at medium r (e.g., dropping to $1.0\times$ at $r=6$), as stricter constraints shrink the search space for enumeration. However, this resilience is transient. At $r=7$, ARB degrades sharply on *soc-pokec* (e.g., taking 2590s at $s=11$

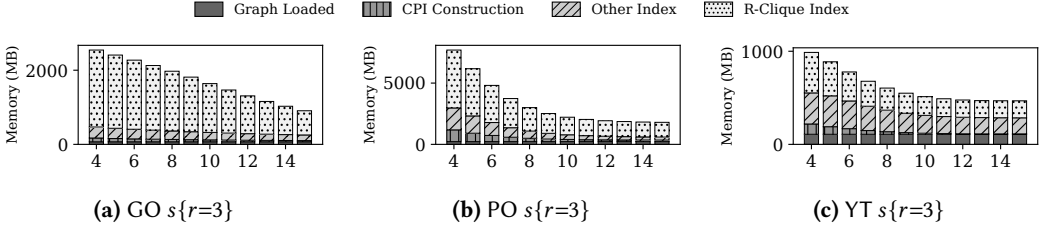


Fig. 9. Breakdown of **memory usage** on different datasets ($r = 3$) as s varies.

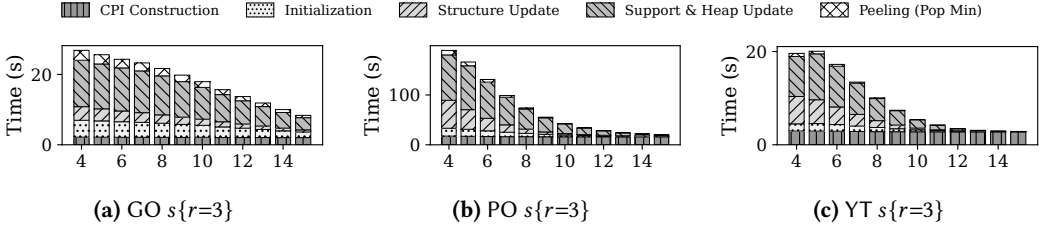


Fig. 10. Breakdown of **running time** on different datasets ($r = 3$) as s varies.

vs. 31s for CND, an $83\times$ difference) and eventually times out. The situation is even worse on com-youtube, where ARB fails to complete entirely at $r \geq 7$. This demonstrates that while enumeration-based frameworks can transiently benefit from filtered search spaces, they remain fundamentally fragile to high-order complexity.

Exp-4: Memory Usage. Figures 8(a-i) and 9 analyze memory consumption. Our method and the state-of-the-art ARB have comparable memory usage overall. As s increases, our memory decreases while ARB stays nearly constant. This is because larger s leaves fewer valid r -cliques: many r -cliques contain no s -cliques. We do not store such r -cliques, whereas ARB cannot reliably pre-filter them and thus maintains states for all r -cliques. For $r=1$, our method uses more memory than ND but is substantially faster. On com-dblp ($r=1, s=4$), ND uses 73MB and takes 62.8s, while our method uses 253MB ($\approx 3.5\times$) and finishes in 0.8s ($77.7\times$ speedup). On web-Stanford ($r=1, s=3$), ND uses 126MB and runs for 596s, while our method uses 656MB and completes in 55s ($\times 10.8\times$ speedups). Thus, even in low-order settings where baselines are optimized, the additional memory translates to significant performance gains. Figure 9 further breaks down CND’s memory usage across datasets and (r, s) settings. Most memory is spent on storing r -cliques rather than CPI, indicating that CPI itself is memory-efficient.

Exp-5: Runtime Breakdown. Figure 10 reports the runtime breakdown of CND across datasets. We decompose the total time into five components: *CPI Construction*, *Init* (initial r -clique enumeration and support counting), *Structure Update* (updating CPI structure during peeling), *Support* (updating s -clique supports via neighbor intersections), and *Peeling* (heap maintenance and PopMin operations). Overall, *Init* and *Support* are the main cost drivers. On com-dblp ($r=4$), *Init* accounts for 36.2%–38.0% and *Support* for 17.7%–18.3% (two runs); on web-Google ($r=4$), *Support* becomes the bottleneck (39.9%) while *Init* is 14.5% (and *Peeling* is 27.3%). These results indicate that runtime is primarily governed by clique enumeration and neighbor intersections (Init/Support), while CPI construction/maintenance and heap-based peeling contribute the remaining overhead. For Q3, the important observation is that *Structure Update* remains a secondary cost across datasets. This is consistent with DPCM: once the initial representation is built, most dynamic work is handled by localized differential updates rather than repeated reconstructive maintenance of the full affected clique space.

Exp-6: Ablation Study for $r \in 1, 2$. Figure 14 reports an ablation study for $r \in \{1, 2\}$. To reduce the overhead of generic clique enumeration at low orders, we add specialized optimizations for $r=1$ and $r=2$. Compared with the generic baseline (CBS3), the optimized version (CBS-noHi) is consistently faster: on com-dblp with $r=2, s=4$, runtime drops from 1.57s to 1.04s ($1.5\times$), and for $r=1$ we observe $\approx 1.3\times$ speedup. Memory usage remains similar since the optimization only changes CPI updates and does not change the stored CPI structure.

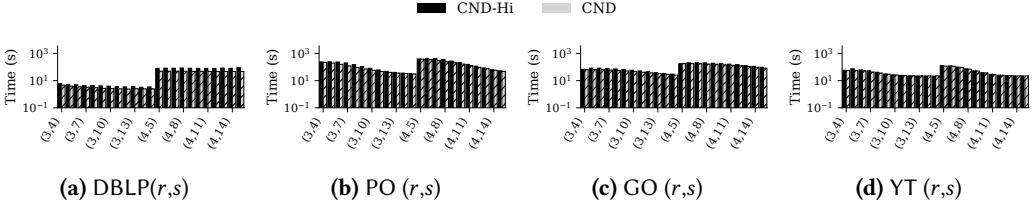
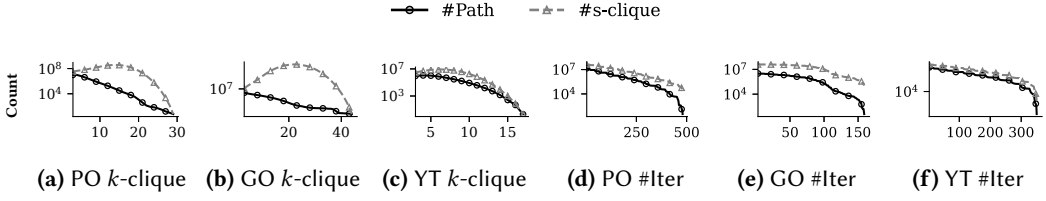
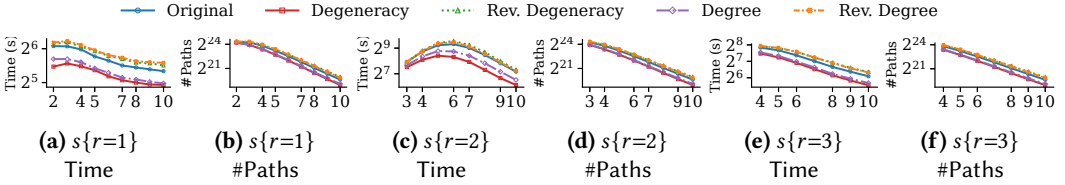
Fig. 11. Time (in seconds) Hierarchical vs Non-Hierarchical with varying r, s .Fig. 12. CPI #Paths vs. #s-cliques for $r = 3, s = 4$.

Fig. 13. Impact of different vertex ordering strategies in soc-pokec

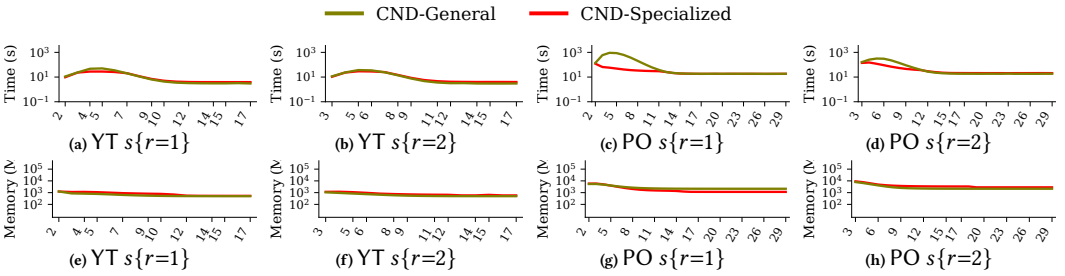


Fig. 14. Comparison of execution time for CND general vs. specialized algorithms.

This directly supports Q2: once the same CPI representation is fixed, regime-specific DPCM operators still contribute additional gains.

Exp-7: Hierarchy vs. No Hierarchy. Figure 11 compares CND with and without hierarchical construction for $r \in \{3, 4\}$ on Youtube, soc-pokec, and web-Google. Across all three datasets, the two variants have nearly identical runtimes; the hierarchical version is about 10% slower on average. This indicates that our CND algorithm exhibits very stable performance regardless of whether hierarchical construction is employed. This confirms Q4: once CPI and DPCM are in place, maintaining s -connectivity for hierarchy adds only modest overhead.

Exp-8: Node Orderings. We evaluate five vertex orderings on soc-pokec (Figure 13): origin, degree/degreeR, and degen/degenR. Across all tested s (2–10 for $r=1$, 3–10 for $r=2$, and 4–10 for $r=3$), degen is consistently the

best overall. Averaged over all tested s , *degen* is $1.39\times$ faster than *origin* for $r=1$ ($52.6s \rightarrow 37.9s$), $1.86\times$ for $r=2$ ($392.5s \rightarrow 210.5s$), and $1.37\times$ for $r=3$ ($139.8s \rightarrow 102.1s$). In contrast, the reverse orderings are consistently the slowest: *degenR/degreeR* are $1.52\text{--}2.11\times$ slower than *degen* on average. Moreover, *degen* shrinks the CPI: compared with *origin*, it reduces CPI #paths by $28.3\% / 30.9\% / 32.5\%$ on average for $r=1/2/3$ (up to 37.2% at $s=10$), translating to average memory savings of $0.82 / 0.67 / 0.37$ GB. For example, at $r=1, s=3$, *degen* reduces CPI #paths from $19.78M$ to $15.82M$ (-20%), lowering memory from 6.35 GB to 5.18 GB (-1.17 GB) and runtime from $67.2s$ to $47.3s$. We therefore adopt *degen* as the default ordering strategy.

Exp-9: #Paths vs. #s-Cliques. Across peeling iterations, we compare the number of CPI paths with the number of s -cliques (Figure 12) for $(r, s) = (3, 4)$ on three datasets. On all datasets and in almost all iterations, the path count is far below $|K_s|$, indicating that CPI stays compact and output-sensitive. Moreover, the path count is flat or decreasing over time, suggesting that path splits are non-inflationary. A mild exception is *com-youtube* at $s=4$, whose sparsity and small maximum clique size ($\omega=17$) narrow the gap, consistent with its more modest speedups (Figure 5h). This experiment isolates the representation side of Q3: even before considering differential updates, CPI already compresses the active clique space substantially.

Exp-10: Stress Test. To evaluate limits under combinatorial explosion, We stress-test on $N=1000$ synthetic graphs built by incremental clique injection: clique sizes are sampled from a truncated power-law (max size 100), and density is increased from $p=0.01$, yielding $\bar{d} \approx p(|V| - 1)$. It is worth noting that our starting density $p=0.01$ is already orders of magnitude higher than typical real-world networks (e.g., *com-dblp* has $p \approx 2 \times 10^{-5}$), creating a rigorous worst-case scenario. Experimental results ($r \in \{4, 5\}, s \in \{5..9\}$) show that *CND* remains robust in these settings. As complexity increases, the baseline *ARB* collapses rapidly: for $(r, s)=(4, 7)$, it times out at a mere $p=0.05$, and for $s \geq 8$, it fails immediately at $p=0.01$. In contrast, *CND* robustly processes densities up to $p=0.20$ across all settings. For $(r, s)=(4, 6)$ at $p=0.11$, *CND* achieves a $210\times$ speedup ($30s$ vs. $6310s$). Regarding memory, although *CND* consumes higher memory than *ARB* ($\approx 2.3\times$), the usage ratio remains constant as density increases. This linear scaling indicates that our index structure effectively withstands the exponential growth of redundant cliques that paralyzes enumeration-based methods.

Leaf cliques vs. total cliques. As p increases, the peak path in CPI count grows and shifts to larger k (e.g., from 999 at $k=3$ when $p=0.01$, to 2.98×10^9 at $k=10$ when $p=0.38$), but the total k -clique count grows much faster and becomes infeasible to enumerate. For example, at $p=0.01$ there are 3.53×10^{10} 20 -cliques but only 46 leaf cliques of size 20 ; at $p=0.38$ there are 8.41×10^{27} 49 -cliques but only $4,028$ leaf cliques of size 49 . This shows that even under extremely dense graphs, CPI still achieves a high compression ratio.

Suggestion for algorithm selection. Our recommendation is simple: *ARB* is competitive for *low-order* settings on *very dense* graphs (e.g., $r=4, s=5, p>0.18$) and uses less memory. In most other regimes—*sparser* graphs ($p<0.18$) or *higher-order* settings ($r \geq 5, s \geq 6$)—we recommend *CND*, which consistently achieves large speedups with comparable memory usage to *ARB*.

Exp-11: Case Study I: Parameter Sensitivity on soc-pokec. We study how r and s control the granularity of (r, s) -nucleus cores on *soc-pokec* (Figure 16). For each core value k , we report the number of surviving vertices and connected components in the remaining nucleus subgraph, and include k -core ($r=1, s=2$) and k -truss ($r=2, s=3$) as baselines. Since the absolute scale of k varies across (r, s) , we compare different settings mainly at the same *rank* of k along each curve.

Varying r with fixed s . With $s=10$, at the 25% - k rank, $r=3$ keeps $141,392$ vertices with 110 components, while $r=6$ keeps $148,295$ vertices but increases to 245 components.

Varying s with fixed r . Fix $r=3$ and increase s from 10 to 14 to 18 . At the 10% - k rank (a typical operating regime), the remaining subgraph shrinks from $195,278$ vertices / 226 components ($s=10$) to $32,310 / 30$ ($s=14$) and further to $5,009 / 8$ ($s=18$). At the median k rank, vertices drop from $78,233 / 41$ ($s=10$) to $11,325 / 14$ ($s=14$) and $2,867 / 5$ ($s=18$).

k -core and k -truss (low-order baselines). At the 10% - k rank, k -core retains $18,449,233$ vertices in a single component, which is too coarse to separate modules. k -truss retains $10,138,083$ vertices but yields $14,286$ components, indicating that low-order $(2, 3)$ support admits many residual structures. By contrast, $(r, s) = (3, 10)$ at the same rank retains $195,278$ vertices and 226 components, producing substantially more compact and well-structured modules.

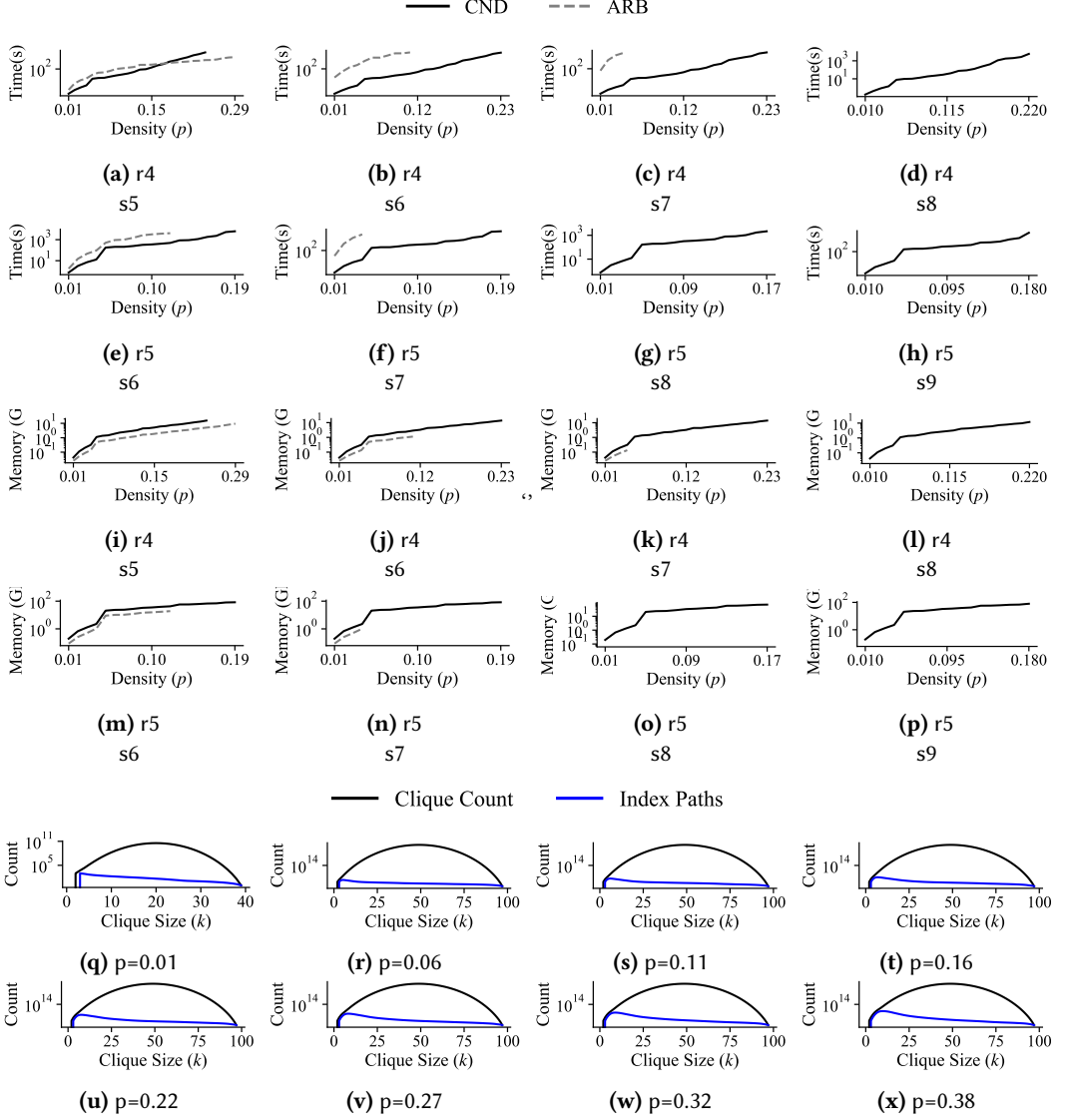


Fig. 15. Comprehensive Stress Test Analysis. (a-h) Runtime comparison across various (r, s) pairs; (i-p) Memory usage comparison corresponding to the top row; (q-x) Leaf clique counts across varying densities (p).

The benefit of tuning s . Increasing s reduces both the surviving size and the number of components, producing compact and well structured modules. It theoretically filters out incidental structures.

Practical guideline. We recommend fixing a moderate r (e.g., $r=3$) and tuning s to control granularity. When s is too large ($s=22$), close to the dataset's maximum clique size (29), very few s -cliques exist and the remaining subgraph may become too small to be informative, most nodes are removed, leaving only a few nodes and components.

Exp-12: Case Study II: Identifying ego network on DBLP. We study Jiawei Han's 1-hop coauthor ego graph on DBLP and compare $s=4$ vs. $s=10$ under $r=3$. In Fig. 17, nodes are colored by nucleus component and

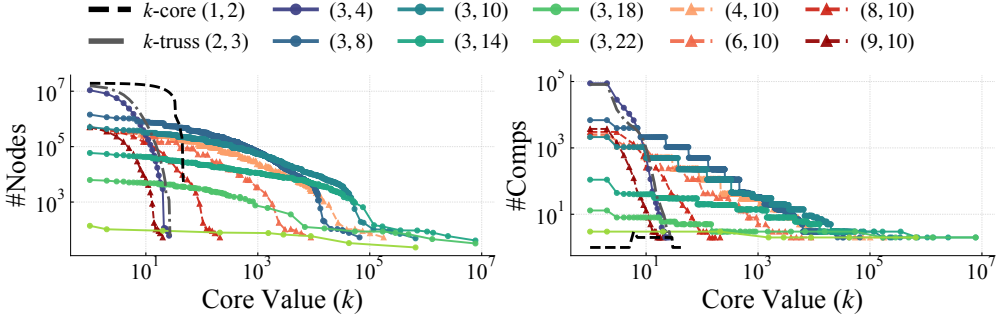


Fig. 16. Case study on soc-pokec. Solid (o): varying s ; dashed (Δ): varying r .

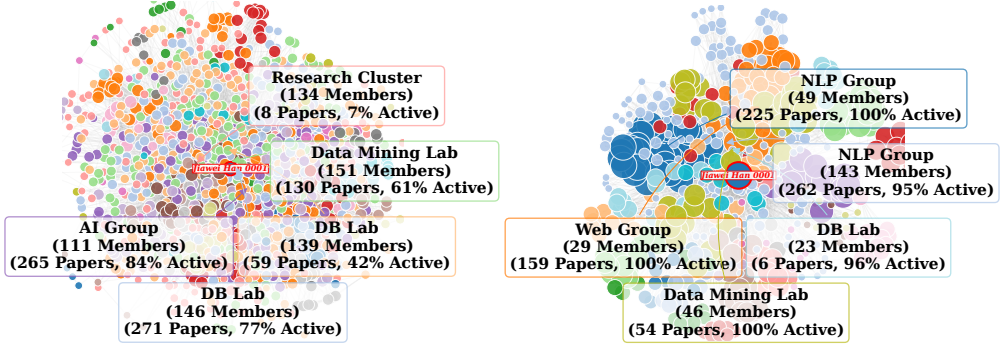


Fig. 17. Case Study on Collaborative network: (a) Low Quality, (b) High Purity.

we annotate the top-5 largest ones. For each surviving component C , let m_{in} be the number of internal edges and m_{cut} the number of edges from C to other surviving components. We report $\phi(C) = m_{cut} / (2m_{in} + m_{cut})$ and m_{in}/m_{cut} ; lower $\phi(C)$ and higher m_{in}/m_{cut} indicate better separation. Using DBLP paper lists restricted to this ego network, a paper is *collaborative* for C if it contains ≥ 3 authors from C , and the *active ratio* is the fraction of members appearing in at least one collaborative paper.

Larger s yields compact and better-separated modules. Increasing s from 4 to 10 shrinks the surviving subgraph from 1,328 to 540 nodes and reduces components from 65 to 27, while increasing the intra-component edge fraction from 0.319 to 0.528. The size-weighted average separability improves from 0.270 to 0.750, and the conductance score drops from 0.712 to 0.475, showing that larger s produces more cleanly separated collaboration modules.

Larger s preserves highly active. Under the same collaboration threshold (3 within-component coauthors), the top-5 annotated components at $s=4$ have an average active ratio of 54.25%, and already exhibit a clear long tail (e.g., a 134-member component has only 8 collaborative papers and 7% active members). In contrast, at $s=10$ the top-8 components have an average active ratio of 98.15%; most are near 100% active, with the smallest 95%. This indicates that increasing s filters loosely coordinated/noisy groups and retains compact, strongly collaborating teams that are easier to interpret.

8 Related Work

Cohesive subgraph discovery is fundamental in graph mining. Classic peeling-based models include k -core decomposition [11, 21, 38, 43, 57, 59, 63–65], which removes vertices with degree $< k$, and k -truss decomposition [1, 8, 18, 50, 55, 56, 58, 66], which removes edges whose triangle support is $< k$. Nucleus decomposition was introduced by [41], generalizing k -core(1, 2) and k -truss(2, 3) to (r, s) -nuclei; other nucleus-decomposition works are discussed earlier and not repeated. At the exact algorithm level, prior nucleus methods mainly differ in how they handle local clique structure during support updates. Some methods maintain explicit clique

incidence. Other methods reconstruct affected local neighborhoods on demand. Still, they share the same high-level peeling skeleton. They also share a reconstructive support-maintenance primitive. Our work differs at this point. We still use the same exact decomposition paradigm. However, we replace reconstructive maintenance by a two-layer design. CPI is the representation layer. DPCM is the exact maintenance layer. Dense-subgraph mining and quasi-clique discovery target objective-driven dense regions or relaxed clique constraints [22, 24]; they are complementary to (r, s) -nuclei, which are defined via higher-order clique containment/support and induce a global hierarchy across core values. Community search is query-dependent, returning a cohesive subgraph containing given query vertices [46, 48]; it emphasizes efficient per-query answers (often under constraints such as size/connectivity), whereas we focus on global (r, s) -nucleus peeling with efficient dynamic index updates. Finally, clique enumeration/counting are key primitives for higher-order cohesiveness; recent work revisits scalable k -clique counting by combining exact counting with sampling [10, 23, 61, 62].

9 Conclusion and future work

In this paper, we revisited nucleus decomposition through two complementary layers. First, we introduced the *Clique Path Index* (CPI), a counting-based representation that eliminates explicit s -clique enumeration. Second, we developed *Differential Path Contribution Maintenance* (DPCM). It updates exact supports by maintaining path-local sufficient states and differential contributions. It does not reconstruct affected clique neighborhoods. Together, these two layers yield exact regime-specific instantiations for $r=1$, $r=2$, and $r \geq 3$. They also preserve efficient hierarchy construction. Extensive experiments show clear speedups for both decomposition and hierarchy construction. They also show that our method scales to larger s .

Our counting-and-indexing viewpoint together with DPCM suggests natural future directions. First, it is amenable to parallelization. BUILDINDEX and INITIALCOUNTING can parallelize over pivot vertices or seed r -cliques with a final reduction. UPDATEINDEX and UPDATESUPPORT can process removed r -cliques in batches by using thread-local buffers and aggregated atomic updates. More importantly, the DPCM formalism suggests a broader design space. This design space includes new sufficient states, symbolic update operators, and regime-specific exact maintenance rules beyond the three instantiations studied here. More broadly, CPI indexes the containment between small patterns and their supporting large patterns. This lets us maintain supports by counting instead of explicit enumeration. This idea extends beyond cliques [13]. Possible targets include bicliques in bipartite graphs (related to (α, β) -core) [51], hyperedge-induced group patterns in hypergraphs [28, 29, 64], and temporal Δ -cliques [9, 54]. We leave optimized multi-threaded implementations, richer DPCM state systems, and these extensions as future work.

10 Acknowledgements

Dong Wen is supported by ARC DP230101445 and ARC DE240100668. Wenjie Zhang is supported by ARC DP230101445 and ARC FT210100303. Xuemin Lin is supported by NSFC U2241211, NSFC U20B2046, and GuangDong Basic and Applied Basic Research Foundation 2019B1515120048.

References

- [1] Esra Akbas and Peixiang Zhao. 2017. Truss-based community search: a truss-equivalence based indexing approach. *Proc. VLDB Endow.* 10, 11 (Aug. 2017), 1298–1309. doi:10.14778/3137628.3137640
- [2] Leman Akoglu, Hanghang Tong, and Danai Koutra. 2015. Graph-based anomaly detection and description: a survey. *Data Mining and Knowledge Discovery* 29, 3 (2015), 626–688.
- [3] V. Batagelj and M. Zaversnik. 2003. An $O(m)$ Algorithm for Cores Decomposition of Networks. arXiv:cs/0310049 [cs.DS] <https://arxiv.org/abs/cs/0310049>
- [4] Vladimir Batagelj and Matjaž Zaveršnik. 2011. Fast algorithms for determining (generalized) core groups in social networks. *Adv. Data Anal. Classif.* 5, 2 (July 2011), 129–145. doi:10.1007/s11634-010-0079-y
- [5] Austin R. Benson, David F. Gleich, and Jure Leskovec. 2016. Higher-order organization of complex networks. *Science* 353, 6295 (July 2016), 163–166. doi:10.1126/science.aad9029
- [6] Alex Beutel, Wanhong Xu, Venkatesan Guruswami, Christopher Palow, and Christos Faloutsos. 2013. CopyCatch: stopping group attacks by spotting lockstep behavior in social networks. In *Proceedings of the 22nd International Conference on World Wide Web* (Rio de Janeiro, Brazil) (WWW '13). Association for Computing Machinery, New York, NY, USA, 119–130. doi:10.1145/2488388.2488400
- [7] Coen Bron and Joep Kerbosch. 1973. Algorithm 457: finding all cliques of an undirected graph. *Commun. ACM* 16, 9 (Sept. 1973), 575–577. doi:10.1145/362342.362367
- [8] Yulin Che, Zhuohang Lai, Shixuan Sun, Yue Wang, and Qiong Luo. 2020. Accelerating truss decomposition on heterogeneous processors. *Proc. VLDB Endow.* 13, 10 (June 2020), 1751–1764. doi:10.14778/3401960.3401971
- [9] Kaiyu Chen, Dong Wen, Wentao Li, Zhengyi Yang, and Wenjie Zhang. 2024. On compressing historical cliques in temporal graphs. In *International Conference on Database Systems for Advanced Applications*. Springer Nature Singapore Singapore, 37–53.
- [10] Kaiyu Chen, Dong Wen, Hanchen Wang, Zhengyi Yang, Wenjie Zhang, and Xuemin Lin. 2025. Covering K-Cliques in Billion-Scale Graphs. In *The Web Conference 2025*.
- [11] James Cheng, Yiping Ke, Shumo Chu, and M. Tamer Özsu. 2011. Efficient core decomposition in massive networks. In *2011 IEEE 27th International Conference on Data Engineering*. International Conference on Data Engineering, Hannover, 51–62. doi:10.1109/ICDE.2011.5767911
- [12] Shumo Chu and James Cheng. 2012. Triangle listing in massive networks. *ACM Trans. Knowl. Discov. Data* 6, 4, Article 17 (Dec. 2012), 32 pages. doi:10.1145/2382577.2382581
- [13] Reinhard Diestel. 2000. *Graph Theory* (2 ed.). Springer.
- [14] David Gibson, Jon Kleinberg, and Prabhakar Raghavan. 1998. Inferring Web communities from link topology. In *Proceedings of the Ninth ACM Conference on Hypertext and Hypermedia: Links, Objects, Time and Space—Structure in Hypermedia Systems: Links, Objects, Time and Space—Structure in Hypermedia Systems* (Pittsburgh, Pennsylvania, USA) (HYPERTEXT '98). Association for Computing Machinery, New York, NY, USA, 225–234. doi:10.1145/276627.276652
- [15] M. Girvan and M. E. J. Newman. 2002. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences* 99, 12 (2002), 7821–7826. arXiv:<https://www.pnas.org/doi/pdf/10.1073/pnas.122653799> doi:10.1073/pnas.122653799
- [16] Wolfgang Glänzel and András Schubert. 2005. *Analysing Scientific Networks Through Co-Authorship*. Springer Netherlands, Dordrecht, 257–276. doi:10.1007/1-4020-2755-9_12
- [17] Bryan Hooi, Hyun Ah Song, Alex Beutel, Neil Shah, Kijung Shin, and Christos Faloutsos. 2016. FRAUDAR: Bounding Graph Fraud in the Face of Camouflage. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Francisco, California, USA) (KDD '16). Association for Computing Machinery, New York, NY, USA, 895–904. doi:10.1145/2939672.2939747
- [18] Xin Huang, Hong Cheng, Lu Qin, Wentao Tian, and Jeffrey Xu Yu. 2014. Querying k-truss community in large and dynamic graphs. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data* (Snowbird, Utah, USA) (SIGMOD '14). Association for Computing Machinery, New York, NY, USA, 1311–1322. doi:10.1145/2588555.2610495
- [19] Shweta Jain and C. Seshadhri. 2020. The Power of Pivoting for Exact Clique Counting. In *Proceedings of the 13th International Conference on Web Search and Data Mining* (Houston, TX, USA) (WSDM '20). Association for Computing Machinery, New York, NY, USA, 268–276. doi:10.1145/3336191.3371839
- [20] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Washington, D.C.) (KDD '03). Association for Computing Machinery, New York, NY, USA, 137–146. doi:10.1145/956750.956769
- [21] Wissam Khaouid, Marina Barsky, Venkatesh Srinivasan, and Alex Thomo. 2015. K-core decomposition of large networks on a single PC. *Proc. VLDB Endow.* 9, 1 (2015), 13–23. doi:10.14778/2850469.2850471
- [22] Aritra Konar and Nicholas D. Sidiropoulos. 2024. Optimal quasi-clique: hardness, equivalence with densest-k-subgraph, and quasi-partitioned community mining. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence*

- and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'24/IAAI'24/AAAI'24). AAAI Press, Article 957, 9 pages. doi:10.1609/aaai.v38i8.28705
- [23] Longbin Lai, Zhu Qing, Zhengyi Yang, Xin Jin, Zhengmin Lai, Ran Wang, Kongzhang Hao, Xuemin Lin, Lu Qin, Wenjie Zhang, et al. 2019. Distributed subgraph matching on timely dataflow. *Proceedings of the VLDB Endowment* (2019).
 - [24] Tommaso Lanciano, Atsushi Miyauchi, Adriano Fazzone, and Francesco Bonchi. 2024. A Survey on the Densest Subgraph Problem and its Variants. *ACM Comput. Surv.* 56, 8, Article 208 (April 2024), 40 pages. doi:10.1145/3653298
 - [25] Jure Leskovec, Kevin J. Lang, Anirban Dasgupta, and Michael W. Mahoney. 2008. Community Structure in Large Networks: Natural Cluster Sizes and the Absence of Large Well-Defined Clusters. arXiv:0810.1355 [cs.DS] <https://arxiv.org/abs/0810.1355>
 - [26] Jure Leskovec and Rok Sosič. 2016. SNAP: A general-purpose network analysis and graph-mining library. *ACM Transactions on Intelligent Systems and Technology* 8, 1 (2016), 1.
 - [27] Greg Linden, Brent Smith, and Jeremy York. 2003. Amazon.com Recommendations: Item-to-Item Collaborative Filtering. *IEEE Internet Computing* 7, 1 (Jan. 2003), 76–80. doi:10.1109/MIC.2003.1167344
 - [28] Qi Luo, Wenjie Zhang, Zhengyi Yang, Dong Wen, Xiaoyang Wang, Dongxiao Yu, and Xuemin Lin. 2024. Hierarchical structure construction on hypergraphs. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 1597–1606.
 - [29] Qi Luo, Wenjie Zhang, Zhengyi Yang, Dongxiao Yu, Xuemin Lin, and Liping Wang. 2025. Efficient indexing and searching of constrained core in hypergraphs. *The VLDB Journal* 34, 3 (2025), 34.
 - [30] Fragkiskos D. Malliaros, Christos Giatsidis, Dimitrios M. Thilikos, and Michalis Vazirgiannis. 2020. The core decomposition of networks: theory, algorithms and applications. *The VLDB Journal* 29, 1 (2020), 61–92. doi:10.1007/s00778-019-00587-4
 - [31] Fragkiskos D. Malliaros and Michalis Vazirgiannis. 2013. Clustering and community detection in directed networks: A survey. *Physics Reports* 533, 4 (2013), 95–142. doi:10.1016/j.physrep.2013.08.002 Clustering and Community Detection in Directed Networks: A Survey.
 - [32] Md Moniruzzaman Monir and Ahmet Erdem Sariyüce. 2020. Using Large Cliques for Hierarchical Dense Subgraph Discovery. In *International Conference on Computational Data and Social Networks*. Springer, 179–192.
 - [33] M. E. J. Newman. 2001. The structure of scientific collaboration networks. *Proceedings of the National Academy of Sciences* 98, 2 (2001), 404–409. arXiv:<https://www.pnas.org/doi/pdf/10.1073/pnas.98.2.404> doi:10.1073/pnas.98.2.404
 - [34] Francis J O'Reilly, Andrea Graziadei, Christian Forbrig, Rica Breckenkamp, Kristine Charles, Swantje Lenz, Christoph Elfmann, Lutz Fischer, Jörg Stülke, and Juri Rappsilber. 2023. Protein complexes in cells by AI‐assisted structural proteomics. *Molecular Systems Biology* 19, 4 (2023), e11544. arXiv:<https://www.embopress.org/doi/pdf/10.15252/msb.202311544> doi:10.15252/msb.202311544
 - [35] Giulio Rossetti and Rémy Cazabet. 2018. Community Discovery in Dynamic Networks: A Survey. *ACM Comput. Surv.* 51, 2, Article 35 (Feb. 2018), 37 pages. doi:10.1145/3172867
 - [36] Ryan A. Rossi and Nesreen K. Ahmed. 2015. The Network Data Repository with Interactive Graph Analytics and Visualization. In *AAAI*. <https://networkrepository.com>
 - [37] Ryan A. Rossi, Nesreen K. Ahmed, Rong Zhou, and Hoda Eldardiry. 2018. Interactive Visual Graph Mining and Learning, Vol. 9. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3200764
 - [38] Ahmet Erdem Sariyüce, Buğra Gedik, Gabriela Jacques-Silva, Kun-Lung Wu, and Ümit V. Çatalyürek. 2013. Streaming algorithms for k-core decomposition. *Proc. VLDB Endow.* 6, 6 (2013), 433–444. doi:10.14778/2536336.2536344
 - [39] Ahmet Erdem Sariyüce and Ali Pinar. 2016. Fast hierarchy construction for dense subgraphs. *Proc. VLDB Endow.* 10, 3 (Nov. 2016), 97–108. doi:10.14778/3021924.3021927
 - [40] Ahmet Erdem Sariyüce, C. Seshadhri, and Ali Pinar. 2018. Local algorithms for hierarchical dense subgraph discovery. *Proc. VLDB Endow.* 12, 1 (Sept. 2018), 43–56. doi:10.14778/3275536.3275540
 - [41] Ahmet Erdem Sariyüce, C. Seshadhri, Ali Pinar, and Umit V. Catalyurek. 2015. Finding the Hierarchy of Dense Subgraphs using Nucleus Decompositions. In *Proceedings of the 24th International Conference on World Wide Web* (Florence, Italy) (WWW '15). International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 927–937. doi:10.1145/2736277.2741640
 - [42] Ahmet Erdem Sariyüce, C. Seshadhri, Ali Pinar, and Ümit V. Çatalyürek. 2017. Nucleus Decompositions for Identifying Hierarchy of Dense Subgraphs. *ACM Trans. Web* 11, 3, Article 16 (July 2017), 27 pages. doi:10.1145/3057742
 - [43] Stephen B. Seidman. 1983. Network structure and minimum degree. *Social Networks* 5, 3 (1983), 269–287. doi:10.1016/0378-8733(83)90028-X
 - [44] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2021. Theoretically and practically efficient parallel nucleus decomposition. *Proc. VLDB Endow.* 15, 3 (Nov. 2021), 583–596. doi:10.14778/3494124.3494140
 - [45] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2024. Parallel Algorithms for Hierarchical Nucleus Decomposition. *Proc. ACM Manag. Data* 2, 1, Article 32 (March 2024), 27 pages. doi:10.1145/3639287

- [46] Mauro Sozio and Aristides Gionis. 2010. The Community-search Problem and How to Plan a Successful Cocktail Party. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. doi:10.1145/1835804.1835923
- [47] K. Subramanyam. 1983. Bibliometric studies of research collaboration: A review. *Journal of Information Science* 6, 1 (1983), 33–38. arXiv:https://doi.org/10.1177/016555158300600105 doi:10.1177/016555158300600105
- [48] Longxu Sun, Xin Huang, Jiannan Wang, and Jianliang Xu. 2025. A Flexible Framework for Query-Oriented Interactive Community Search. *Proc. VLDB Endow.* 18, 6 (Feb. 2025), 1977–1990. doi:10.14778/3725688.3725720
- [49] Lubos Takac. 2012. DATA ANALYSIS IN PUBLIC SOCIAL NETWORKS. <https://api.semanticscholar.org/CorpusID:18659578>
- [50] Anxin Tian, Alexander Zhou, Yue Wang, and Lei Chen. 2023. Maximal D-Truss Search in Dynamic Directed Graphs. *Proc. VLDB Endow.* 16, 9 (May 2023), 2199–2211. doi:10.14778/3598581.3598592
- [51] Anxin Tian, Alexander Zhou, Yue Wang, Xun Jian, and Lei Chen. 2024. Efficient Index for Temporal Core Queries over Bipartite Graphs. *Proc. VLDB Endow.* 17, 11 (July 2024), 2813–2825. doi:10.14778/3681954.3681965
- [52] Etsuji Tomita, Akira Tanaka, and Haruhisa Takahashi. 2006. The worst-case time complexity for generating all maximal cliques and computational experiments. *Theor. Comput. Sci.* 363, 1 (Oct. 2006), 28–42. doi:10.1016/j.tcs.2006.06.015
- [53] Johan Ugander, Lars Backstrom, Cameron Marlow, and Jon Kleinberg. 2012. Structural diversity in social contagion. *Proceedings of the National Academy of Sciences* 109, 16 (2012), 5962–5966. arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.1116502109 doi:10.1073/pnas.1116502109
- [54] Tiphaine Viard, Matthieu Latapy, and Clémence Magnien. 2016. Computing maximal cliques in link streams. *Theor. Comput. Sci.* 609, P1 (Jan. 2016), 245–252. doi:10.1016/j.tcs.2015.09.030
- [55] Jia Wang and James Cheng. 2012. Truss decomposition in massive networks. *Proc. VLDB Endow.* 5, 9 (May 2012), 812–823. doi:10.14778/2311906.2311909
- [56] Yue Wang, Ruiqi Xu, Xun Jian, Alexander Zhou, and Lei Chen. 2022. Towards distributed bitruss decomposition on bipartite graphs. *Proc. VLDB Endow.* 15, 9 (May 2022), 1889–1901. doi:10.14778/3538598.3538610
- [57] Dong Wen, Lu Qin, Ying Zhang, Xuemin Lin, and Jeffrey Xu Yu. 2016. I/O efficient core graph decomposition at web scale. In *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*. IEEE, 133–144.
- [58] Tianyang Xu, Zhao Lu, and Yuanyuan Zhu. 2022. Efficient Triangle-Connected Truss Community Search in Dynamic Graphs. *Proc. VLDB Endow.* 16, 3 (Nov. 2022), 519–531. doi:10.14778/3570690.3570701
- [59] Bohua Yang, Dong Wen, Lu Qin, Ying Zhang, Lijun Chang, and Rong-Hua Li. 2019. Index-Based Optimal Algorithm for Computing K-Cores in Large Uncertain Graphs. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. 64–75. doi:10.1109/ICDE.2019.00015
- [60] Jaewon Yang and Jure Leskovec. 2012. Defining and Evaluating Network Communities based on Ground-truth. arXiv:1205.6233 [cs.SI] <https://arxiv.org/abs/1205.6233>
- [61] Zhengyi Yang, Longbin Lai, Xuemin Lin, Kongzhang Hao, and Wenjie Zhang. 2021. Huge: An efficient and scalable subgraph enumeration system. In *Proceedings of the 2021 international conference on management of data*. 2049–2062.
- [62] Xiaowei Ye, Rong-Hua Li, Qiangqiang Dai, Hongzhi Chen, and Guoren Wang. 2022. Lightning Fast and Space Efficient k-clique Counting. In *Proceedings of the ACM Web Conference 2022 (Virtual Event, Lyon, France) (WWW '22)*. Association for Computing Machinery, New York, NY, USA, 1191–1202. doi:10.1145/3485447.3512167
- [63] Michael Yu, Dong Wen, Lu Qin, Ying Zhang, Wenjie Zhang, and Xuemin Lin. 2021. On Querying Historical K-Cores. *Proc. VLDB Endow.* 14, 11 (2021), 2033–2045. doi:10.14778/3476249.3476260
- [64] Wenqian Zhang, Zhengyi Yang, Dong Wen, Wentao Li, Wenjie Zhang, and Xuemin Lin. 2025. Accelerating Core Decomposition in Billion-Scale Hypergraphs. *Proc. ACM Manag. Data* 3, 1, Article 6 (Feb. 2025), 27 pages. doi:10.1145/3709656
- [65] Wenqian Zhang, Zhengyi Yang, Dong Wen, and Xiaoyang Wang. 2023. Efficient Distributed Core Graph Decomposition. In *2023 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 1023–1031.
- [66] Feng Zhao and Anthony K. H. Tung. 2012. Large scale cohesive subgraphs discovery for social network visual analysis. *Proc. VLDB Endow.* 6, 2 (Dec. 2012), 85–96. doi:10.14778/2535568.2448942
- [67] Zhengzhang Zhu, Chen Lin, Xuejian Song, Jiuxin Cao, Wei Wang, and Albert Y. Zomaya. 2021. Efficient truss decomposition for large-scale graphs. *IEEE Transactions on Knowledge and Data Engineering* 33, 10 (2021), 3343–3356. doi:10.1109/TKDE.2020.2978311